

银犁智慧配送

从零制作与完整复现手册

从一个空文件夹开始，完整搭建当前网站。不是客户下单操作说明，而是开发、数据、测试和部署教程。

目标	只凭本 PDF 与规定的原始数据，重新创建目录、安装依赖、编写全部模块和页面、生成数据、启动并验收网站。
对应版本	银犁智慧配送当前 Streamlit 本地版本，2026-09-17。
关键边界	客户地图蓝线是订单演示调度线；正式路网最短路径和优化结果来自竞赛成果数据，二者不得混称。

正文按实际制作顺序编排；遇到“创建某文件”时，从文末代码附录复制该文件。数据文件体积较大，不在 PDF 中逐字展开，而由脚本生成。

内容导航

内容导航	2
第一章：明确要复现的网站架构	5
1.1 六层结构	5
1.2 数据流与业务流	5
1.3 技术栈	5
第二章：从空目录建立环境	6
2.1 安装 Python 与验证	6
2.2 建立项目目录和虚拟环境	6
2.3 创建目录树	6
2.4 依赖安装	6
2.5 Streamlit 配置	6
第三章：准备网站数据	7
3.1 三类数据	7
3.2 原始成果目录约定	7
3.3 生成步骤	7
3.4 数据文件契约	7
第四章：先实现公共模块	8
4.1 UI 与导航	8
4.2 订单状态与隐私	8
4.3 报价	8
4.4 地址与地图	8
4.5 快速初算与正式模型	9
第五章：逐页构建网站	10
5.1 app.py 首页	10
5.2 客户下单	10
5.3 订单追踪	10
5.4 客户配送地图	10
5.5 电子签收	10
5.6 企业工作台	10
5.7 运营配送地图	11
5.8 模型成果页	11
第六章：把订单闭环接通	12

6.1 状态动作矩阵	12
6.2 演示调度线	12
6.3 本地持久化	12
6.4 地址识别的容错	12
第七章：启动入口	13
7.1 首次手工启动	13
7.2 双击脚本与快捷方式	13
7.3 首次打开检查	13
第八章：测试与验收	14
8.1 静态与数据测试	14
8.2 完整业务回归	14
8.3 HTTP 路由	14
8.4 视觉验收	14
第九章：迁移和公网部署	15
9.1 迁移到新电脑	15
9.2 Streamlit Community Cloud	15
第十章：故障排查与最终清单	16
10.1 最终复现验收清单	16
10.2 最小交付包	16
代码附录：requirements.txt	18
代码附录：runtime.txt	19
代码附录：.streamlit/config.toml	20
代码附录：app.py	21
代码附录：功能组件_页面共用代码/_init_.py	23
代码附录：功能组件_页面共用代码/data_loader.py	24
代码附录：功能组件_页面共用代码/order_state.py	26
代码附录：功能组件_页面共用代码/maps.py	29
代码附录：功能组件_页面共用代码/planner.py	31
代码附录：功能组件_页面共用代码/matlab_bridge.py	35

代码附录：功能组件_页面共用代码/ui.py	36
代码附录：pages/客户下单.py	41
代码附录：pages/订单追踪.py	43
代码附录：pages/配送网络地图.py	44
代码附录：pages/电子签收.py	45
代码附录：pages/企业工作台.py	46
代码附录：pages/运营配送地图.py	47
代码附录：pages/方案驾驶室.py	48
代码附录：pages/订单与需求.py	49
代码附录：pages/车辆路径优化.py	50
代码附录：pages/成本与绩效.py	51
代码附录：pages/方案对比.py	52
代码附录：pages/数据导入与方案生成.py	53
代码附录：数据整理脚本_生成网站数据/prepare_data.py	55
代码附录：数据整理脚本_生成网站数据/export_road_network.py	58
代码附录：数据核验测试_检查结果/test_data_contract.py	59
代码附录：README.md	60
代码附录：启动网站.bat	61
代码附录：打开本地网站.url	62

第一章：明确要复现的网站架构

该项目是 Streamlit 多页面 Python 应用，不是静态 HTML。浏览器负责展示，Python 负责表单、订单状态、报价、地图、文件读取和规划逻辑。

1.1 六层结构

层	文件或目录	职责
入口	app.py、启动网站.bat、打开本地网站.url	启动与首页
客户业务	客户下单、订单追踪、配送网络地图、电子签收	客户订单闭环
工作人员	企业工作台、运营配送地图	全量订单与状态推进
决策展示	驾驶舱、订单与需求、路径、成本、对比	竞赛结果展示
公共能力	ui、order_state、maps、data_loader、planner、m atlab_bridge	复用逻辑
数据与测试	网站数据、整理脚本、契约测试	派生与验证

1.2 数据流与业务流

1. 客户提交表单。
2. order_state.py 生成订单号、费用、状态与演示调度线。
3. 客户追踪、地图和签收只读取当前会话订单。
4. 工作人员验证环境变量密码后读取本机 Excel 订单。
5. 工作人员按状态机逐级推进。
6. 配送途中后客户才能签收。
7. 模型展示页面独立读取网站数据目录。

客户临时订单与竞赛模型成果是两个数据域。前者支撑现场交互，后者支撑正式模型展示；复现时不要互相替代。

1.3 技术栈

技术	用途	选择原因
Python 3.11-3.13	服务端逻辑	Windows 和 Streamlit 兼容稳定
Streamlit	页面、表单、状态	快速构建多页面演示系统
Pandas/OpenPyXL	CSV、Excel、订单	适配竞赛成果格式
Plotly	交互图表	嵌入方便
Folium/streamlit-folium	地图	支持本地 GeoJSON
GeoPandas/NetworkX	预处理 GPKG 与最短路	仅数据生成阶段需要

第二章：从空目录建立环境

2.1 安装 Python 与验证

1. 安装 64 位 Python 3.11、3.12 或 3.13，并勾选 Add Python to PATH。
2. 打开 PowerShell，执行 `python --version`。
3. 执行 `python -m pip --version`。
4. 若电脑有多个 Python，后续始终使用同一个 python 命令。

2.2 建立项目目录和虚拟环境

1. 创建网站根目录。
2. 在根目录打开 PowerShell。
3. 执行 `python -m venv .venv`。
4. 执行 `.\venv\Scripts\Activate.ps1`。
5. 若执行策略阻止激活，先执行 `Set-ExecutionPolicy -Scope Process Bypass`。

2.3 创建目录树

目录	内容	是否自动产生
pages	全部子页面	否
功能组件_页面共用代码	公共模块	否
网站数据_页面读取的指标	轻量 JSON/CSV/GeoJSON	脚本写入
数据整理脚本_生成网站数据	数据派生脚本	否
数据核验测试_检查结果	数据契约测试	否
展示图片_地图和答辩图	成果图片	否
客户订单数据	本地订单 Excel	首次下单自动创建
.streamlit	主题与服务器配置	否

2.4 依赖安装

1. 创建 requirements.txt，内容见附录。
2. 执行 `python -m pip install --upgrade pip`。
3. 执行 `python -m pip install -r requirements.txt`。
4. 如需从 GPKG 重建路网，再安装 `geopandas networkx pyogrio`。
5. 执行 `python -c "import streamlit,pandas,plotly,folium,openpyxl; print('OK')"`。

2.5 Streamlit 配置

创建 `.streamlit/config.toml`，配置浅色主题、橙色主按钮、200 MB 上传、XSRF 防护和关闭使用统计。工作人员密码不得写入 Python 或 TOML，应通过 `YL_STAFF_PASSWORD` 或 Streamlit secrets 提供。

第三章：准备网站数据

3.1 三类数据

类型	来源	用途
原始成果	完整模型 JSON/CSV/XLSX/GPKG	生成轻量数据
网站展示数据	prepare_data.py 派生	驾驶舱、图表、地图
客户演示订单	网页表单	追踪、签收、工作台

3.2 原始成果目录约定

变量	默认目录或文件	需要内容
NOODLE_DIR	../__AAAAA面条优化结果_完整模型版	结果 JSON、路线/到达 CSV、客户表
GINGER_DIR	../__AAAAA3.2姜蒜专线优化结果_完整模型版	结果 JSON、路线/到达 CSV、客户表
REUSE_DIR	../__AAAAA第二问优化结果_增加车辆复用版	复用 JSON 与时刻表
SENS_DIR	../__载重对续航的影响（灵敏度分析）	灵敏度 CSV
GPKG	../7.0-修复孤立客户拓扑节点.gpkg	lines、客户点、配送中心图层

3.3 生成步骤

1. 复制附录中的 prepare_data.py 和 export_road_network.py。
2. 按本机成果包位置修改脚本顶部路径常量。
3. 执行 python 数据整理脚本_生成网站数据\prepare_data.py。
4. 需要重建主次道路时执行 export_road_network.py。
5. 检查网站数据目录是否出现 summaries、history、manifest、customers、routes、arrivals、reuse、sensitivity 和 GeoJSON。

3.4 数据文件契约

文件	关键结构	使用处
summaries.json	noodle/ginger/reuse 指标与成本	驾驶舱、成本、对比
algorithm_history.json	搜索历史	路径优化
customers.csv	编号、坐标、产品、需求、时间窗	订单与需求、地图
routes_*.csv	车辆、访问序列、里程、载重	路线页面
arrivals_*.csv	到达、服务、准时	需求页面
reuse_schedule.csv	车辆、任务、起止时刻	驾驶舱
sensitivity.csv	参数与成本	成本页
customer_points.geojson	Point FeatureCollection	地图
route_features.geojson	LineString FeatureCollection	正式路线
road_major/minor.geojson	主次道路	离线底图

完整 GPKG 不直接在网页运行时解析。数据准备阶段把它变成轻量 GeoJSON，避免网页启动慢、云端体积过大。

第四章：先实现公共模块

模块	职责	关键约束
ui.py	CSS、组件、侧栏、权限	导航文件名必须真实；图标风格统一
data_loader.py	集中读取与缓存	缺文件记录 errors，不让全站崩溃
maps.py	本地路网和路线图层	tiles=None，避免在线底图依赖
order_state.py	订单、报价、状态、存储	客户会话隔离；编号防碰撞
planner.py	上传与快速初算	不得冒充正式算法
matlab_bridge.py	调用已核验 MATLAB	路径和超时必须明确

4.1 UI 与导航

1. 定义蓝色品牌主色、橙色动作色、灰白背景、圆角、阴影、间距和字号。
2. 通过 inject_global_css 统一 Streamlit 控件。
3. render_sidebar 始终展示客户入口。
4. 读取环境变量密码后才展示工作人员入口。
5. 工作人员地图指向运营配送地图，不指向客户单订单地图。
6. 用统一线性符号，避免混杂彩色 emoji。

4.2 订单状态与隐私

1. 状态机固定为订单已提交、仓库备货中、等待装车、配送途中、配送完成。
2. staff_password 先读环境变量，再安全读取 secrets；缺文件时返回 None。
3. init_orders() 只返回 session_state 当前订单。
4. 工作人员页面显式调用 init_orders(include_saved=True)。
5. create_order 使用完整时间戳加 UUID 短尾码。
6. persist_order 更新同一订单，不重复追加。
7. advance_order 一次只推进一级。

4.3 报价

预估费用 = 起步价 + 重量×品类基础价 + 参考距离×里程价。默认起步价 12 元、里程价 0.8 元/公里、鲜面条 0.75 元/千克、姜蒜 0.41 元/千克。该报价只用于网页透明展示，不等同于正式模型总成本函数。

4.4 地址与地图

1. 地理编码只在用户点击按钮后调用。
2. 结果用 st.cache_data 缓存 24 小时。
3. 识别失败时允许手工经纬度。
4. Folium 地图禁用在线瓦片并加载本地 road_major/minor。
5. 订单蓝线只连接配送中心和客户坐标，界面必须称为演示调度线。
6. 正式路线来自 route_features.geojson。

4.5 快速初算与正式模型

维度	快速初算	正式模型
输入	上传客户坐标和需求	完整路网、车辆与约束
距离	坐标近似	道路最短路
算法	快速启发式	团队核验程序
结论	演示即时方案	论文和答辩依据
表述	快速可行初算	较优可行解，不声称全局最优

第五章：逐页构建网站

5.1 app.py 首页

1. `st.set_page_config` 必须最先执行，侧栏默认 `expanded`。
2. 调用统一 CSS 与侧栏。
3. 只放品牌、价值、三步流程和入口。
4. `page_link` 必须与真实文件名一致。
5. 首页不要加载大表格和地图。

5.2 客户下单

1. 采集客户、联系人、电话和地址。
2. 地址识别放在按钮事件；保留手填坐标。
3. 品类、重量和时间窗必填。
4. 默认时间窗 08:00-18:00，并校验先后。
5. 提交前展示费用拆分。
6. 坐标齐全后调用 `create_order`。

5.3 订单追踪

1. 只调用 `init_orders()`。
2. 无订单显示空状态。
3. 展示状态、时间窗、费用和配送安排。
4. 状态顺序来自 `STATUS_FLOW`。

5.4 客户配送地图

1. 只读取当前会话订单。
2. 绘制配送中心、客户点、本地道路。
3. 叠加订单蓝色演示线。
4. 明确说明不等同正式最短路。

5.5 电子签收

1. 状态必须严格等于配送途中才显示按钮。
2. 确认后持久化配送完成。
3. 异常反馈独立保存。
4. 完成后不重复确认。

5.6 企业工作台

1. 开头 `require_staff_access`。
2. 读取 `include_saved=True`。
3. 显示订单指标。
4. 一次点击只推进一级。
5. 演示边界集中成一条提示。

5.7 运营配送地图

1. 仅工作人员可见。
2. 显示正式路网与全部本机订单。
3. 用颜色区分正式路线和演示线。

5.8 模型成果页

1. 驾驶舱展示关键指标与总览。
2. 订单与需求展示客户和到达。
3. 路径页展示车辆路线和搜索历史。
4. 成本页展示拆分与敏感性。
5. 对比页横向比较三类方案。
6. 数据导入页提供上传、校验、快速初算和下载。

第六章：把订单闭环接通

6.1 状态动作矩阵

状态	追踪	地图	工作人员	签收
订单已提交	可见	可能无路线	推进	禁止
仓库备货中	可见	有演示线	推进	禁止
等待装车	可见	保留	推进	禁止
配送途中	可见	保留	可推进	允许
配送完成	完成	历史展示	停止	不重复

6.2 演示调度线

下单有经纬度时，`create_order` 同步写入 `FeatureCollection/LineString`，坐标顺序为配送中心到客户。该线解决“下单后地图永远无路线”的流程断点，但它只用于串联演示流程。正式 Dijkstra 路线继续使用竞赛成果数据。

6.3 本地持久化

订单保存到 客户订单数据/日期/品类订单.xlsx。路线和报价字典写入前转成 JSON 字符串，读取时恢复。客户页面不扫描该目录；工作人员页面经授权后读取。

6.4 地址识别的容错

1. 用户输入地址。
2. 点击识别按钮才发起 Nominatim 请求。
3. 成功后回填坐标。
4. 失败时显示提示但不丢失表单。
5. 手填坐标后仍可完成报价、下单、地图、追踪和签收。

第七章：启动入口

7.1 首次手工启动

1. 进入根目录并激活 .venv。
2. 执行 `$env:YL_STAFF_PASSWORD='设置自己的演示密码'`。
3. 执行 `python -m streamlit run app.py --server.headless true --server.port 8501`。
4. 访问 `http://localhost:8501/`。
5. 保持终端打开，Ctrl+C 停止。

7.2 双击脚本与快捷方式

启动网站.bat 负责进入正确目录、配置本地环境、启动服务并等待端口；打开本地网站.url 只负责访问 `http://localhost:8501/`，不能代替启动服务。公开仓库不得包含真实密码。

7.3 首次打开检查

- 首页标题正确。
- 侧栏默认展开。
- 客户入口可点击。
- 工作人员入口未验证前隐藏。
- 页面无 Python 异常红框。
- 控制台无 `ModuleNotFoundError`。

第八章：测试与验收

8.1 静态与数据测试

1. 对 app.py、pages 和公共模块执行 py_compile。
2. 搜索硬编码密码。
3. 搜索旧数字前缀与无效 page_link。
4. 运行 test_data_contract.py。
5. 缺文件先重新生成数据，缺字段再修正映射。

8.2 完整业务回归

编号	动作	通过条件
C01	打开首页	HTTP 200，侧栏展开
C02	完整下单并填坐标	生成唯一编号和报价
C03	追踪	仓库备货中，已有配送安排
C04	客户地图	出现中心、客户点、蓝线
C05	备货阶段打开签收	无确认按钮
C06	工作人员逐级推进	每次只前进一步
C07	配送途中签收	完成并持久化
C08	新会话查看客户订单	不显示他人历史订单
C09	断网手填坐标	闭环仍能完成

8.3 HTTP 路由

使用 Invoke-WebRequest

逐一请求首页、客户下单、订单追踪、配送网络地图、电子签收和运营配送地图，StatusCode 应为 200。HTTP 200 不能替代交互测试。

8.4 视觉验收

- 1920×1080 无横向滚动。
- 蓝橙品牌色、圆角和阴影一致。
- 服务图标统一为成熟线性符号。
- 表单标签与错误信息对齐。
- 地图和图表不截断。
- 主按钮与次按钮层级清楚。

第九章：迁移和公网部署

9.1 迁移到新电脑

1. 复制源码、轻量网站数据和必要图片，不复制 .venv。
2. 重新安装 Python、创建虚拟环境并安装依赖。
3. 设置密码环境变量。
4. 运行数据契约测试。
5. 启动并执行完整业务回归。
6. 如需 MATLAB，修改本机可执行文件与模型目录。

9.2 Streamlit Community Cloud

1. 以网站目录为 GitHub 仓库根目录。
2. 提交源码、requirements、runtime、轻量数据和必要图片。
3. 排除订单、密码、个人信息、大型路网、旧备份和 .venv。
4. 在 Streamlit Cloud 选择 app.py。
5. 在 Secrets 配置 YL_STAFF_PASSWORD。
6. 部署后检查路径大小写、中文编码和地图体积。

云端本地磁盘会重置。当前 Excel

持久化适合本地竞赛演示，不是生产数据库。正式上线必须接入有权限控制的外部数据库或订单服务。

第十章：故障排查与最终清单

现象	原因	检查	修复
无法打开	服务未启动	请求 localhost:8501	先运行启动脚本
8501 占用	旧进程	netstat -ano findstr :8501	确认 PID 后关闭
导航报错	文件名不一致	核对 pages 与 ui.py	统一路径
客户见历史单	错误读取 saved	搜索 init_orders	客户页不传 True
地图无蓝线	无坐标/路线	查经纬度和路线GeoJSON	提交前强制坐标
提前签收	状态条件太宽	查电子签收判断	只允许配送途中
地址输入卡	每次 rerun 请求	查调用位置	按钮触发并缓存
离线地图空	在线瓦片/缺文件	断网检查数据	tiles=None 加本地路网
云端订单丢	临时磁盘	查重启日志	接外部数据库

10.1 最终复现验收清单

序号	验收项	完成
1	虚拟环境和依赖可重建	☐
2	目录结构完整	☐
3	公共模块可导入	☐
4	数据契约测试通过	☐
5	全部页面 HTTP 200	☐
6	下单生成唯一编号与演示线	☐
7	客户会话隔离	☐
8	工作人员逐级推进	☐
9	配送途中前禁止签收	☐
10	断网地图可显示	☐
11	快速初算与正式模型边界清楚	☐
12	发布包不含密码和客户数据	☐

10.2 最小交付包

- app.py、pages、公共模块。
- requirements、runtime、Streamlit 配置。
- 轻量网站数据与必要图片。
- 数据整理脚本和契约测试。
- README、启动脚本、网址快捷方式。
- 本 PDF。

完整源码附录

下面自动读取当前版本源文件。按标题路径保存，全部 Python 文件使用 UTF-8。大型数据不展开，按第三章生成。

代码附录：requirements.txt

用途：运行依赖。当前版本共 6 行。

```
0001 streamlit
0002 pandas
0003 plotly
0004 folium
0005 streamlit-folium
0006 openpyxl
```

代码附录：runtime.txt

用途：云端 Python 版本。当前版本共 1 行。

0001 python-3.12

代码附录: .streamlit/config.toml

用途: 主题与服务器配置。当前版本共 16 行。

```
0001 [theme]
0002 base = "light"
0003 primaryColor = "#ff8133"
0004 backgroundColor = "#f5f7fb"
0005 secondaryBackgroundColor = "#eef4ff"
0006 textColor = "#1d2a3a"
0007 font = "sans serif"
0008
0009 [server]
0010 headless = true
0011 enableCORS = false
0012 enableXsrfProtection = true
0013 maxUploadSize = 200
0014
0015 [browser]
0016 gatherUsageStats = false
```

代码附录：app.py

用途：首页。当前版本共 86 行。

```
0001 import streamlit as st
0002
0003 from 功能组件_页面共用代码.ui import inject_css, render_sidebar
0004
0005 st.set_page_config(page_title="银犁智慧配送", page_icon="YL", layout="wide", initial_sidebar_state="expanded")
0006 inject_css()
0007 render_sidebar()
0008
0009 st.markdown("""
0010 <div class="home-nav">
0011   <div class="home-brand"><span>YL</span><strong>银犁智慧配送</strong></div>
0012   <div class="home-nav-copy">让每一次配送都有清楚的答案</div>
0013 </div>
0014 """, unsafe_allow_html=True)
0015
0016 with st.container(key="home_hero"):
0017     hero_copy, hero_visual = st.columns([1.08, 0.92], gap="large", vertical_alignment="center")
0018     with hero_copy:
0019         st.markdown("""
0020         <div class="home-hero-copy">
0021           <h1>下单、追踪、签收，一次完成。</h1>
0022           <p>填写地址、货量和送达时间，随时查看配送进度与您的专属路线。</p>
0023         </div>
0024         """, unsafe_allow_html=True)
0025         action_a, action_b = st.columns(2)
0026         with action_a:
0027             if st.button("立即下单", type="primary", use_container_width=True, key="hero_upload"):
0028                 st.switch_page("pages/客户下单.py")
0029         with action_b:
0030             if st.button("查询订单", use_container_width=True, key="hero_map"):
0031                 st.switch_page("pages/订单追踪.py")
0032         st.caption("目前支持鲜面条和姜蒜配送，费用在提交前清楚展示。")
0033     with hero_visual:
0034         st.markdown("""
0035         <div class="customer-journey motion-focus">
0036           <div><span>01</span><b>提交配送需求</b><small>地址、品类、重量和时间</small></div>
0037           <div><span>02</span><b>查看配送进度</b><small>从接单到送达全程可查</small></div>
0038           <div><span>03</span><b>确认收货</b><small>一键签收或提交异常反馈</small></div>
0039         </div>
0040         """, unsafe_allow_html=True)
0041
0042 st.markdown("""
0043 <section class="home-section home-reveal">
0044   <h2>您关心的配送信息，都在这里。</h2>
0045   <p>从提交需求到确认收货，每一步都有清楚的状态和下一步提示。</p>
0046 </section>
0047 <div class="home-bento">
0048   <article class="home-feature home-feature-large home-reveal"><h3>填写一次，配送需求清楚送达。</h3><p>客户名称、联系人、地址、电话、品类、重量和送达时间集中填写，提交前即可查看预估费用。</p><div class="feature-line"></div></article>
0049   <article class="home-feature home-feature-blue home-reveal"><h3>进度随时可查</h3><p>订单已提交、仓库备货、等待装车、配送途中和配送完成，状态一目了然。</p></article>
0050   <article class="home-feature home-feature-light home-reveal"><h3>只看自己的路线</h3><p>提交收货坐标后即可在客户地图查看本订单的演示调度线与配送进度。</p></article>
0051   <article class="home-feature home-feature-dark home-reveal"><h3>签收更简单</h3><p>正常到货一键确认；有包装、数量、温度或延误问题，可直接提交异常反馈。</p></article>
0052 </div>
0053 """, unsafe_allow_html=True)
0054
0055 with st.container(key="home_flow"):
0056     st.markdown("""
0057     <section class="home-section home-reveal"><h2>三步完成一次配送。</h2><p>无需上传表格，客户只需填写自己的配送需求。</p></section>
0058     <div class="home-flow">
0059       <div class="home-flow-item home-reveal"><b>下单</b><span>填写地址、品类、重量和送达时间</span></div>
0060       <div class="home-flow-item home-reveal"><b>追踪</b><span>查看备货、装车和配送进度</span></div>
0061       <div class="home-flow-item home-reveal"><b>签收</b><span>确认收货，异常情况及时反馈</span></div>
0062     </div>
0063     """, unsafe_allow_html=True)
0064
0065 st.markdown("<section class='home-section home-reveal'><h2>按您关心的内容直接进入。</h2></section>", unsafe_allow_html=True)
0066 entry_cols = st.columns(4, gap="medium")
0067 entries = [
0068     ("pages/客户下单.py", "客户下单", "提交配送需求", "□"),
0069     ("pages/订单追踪.py", "订单追踪", "查看处理进度", "●"),
0070     ("pages/配送网络地图.py", "配送地图", "看位置与路线", "◇"),
0071     ("pages/电子签收.py", "电子签收", "确认收货或反馈异常", "□"),
0072 ]
0073 for col, (page, title, copy, icon) in zip(entry_cols, entries):
0074     with col:
0075         with st.container(key=f"entry_{title}"):
0076             st.markdown(f"<div class='entry-icon'>{icon}</div>", unsafe_allow_html=True)
0077             st.page_link(page, label=title, use_container_width=True)
0078             st.caption(copy)
0079
0080 st.markdown("""
```

```
0081 <div class="home-final home-reveal">
0082 <div><h2>准备好配送信息，就可以开始下单。</h2><p>提交前显示预估费用，提交后可随时查看进度。</p></div>
0083 </div>
0084 """, unsafe_allow_html=True)
0085 if st.button("填写配送订单", type="primary", use_container_width=True, key="final_upload"):
0086     st.switch_page("pages/客户下单.py")
```

代码附录：功能组件_页面共用代码/__init__.py

用途：包标记。当前版本共 1 行。

```
0001 """网站共用组件。"""
```

代码附录：功能组件_页面共用代码/data_loader.py

用途：数据加载。当前版本共 92 行。

```
0001 from __future__ import annotations
0002
0003 import json
0004 from dataclasses import dataclass, field
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import pandas as pd
0009 import streamlit as st
0010
0011
0012 ROOT = Path(__file__).resolve().parents[1]
0013 DATA_DIR = ROOT / "网站数据_页面读取的指标"
0014
0015
0016 @dataclass
0017 class SiteData:
0018     summary: dict[str, Any] = field(default_factory=dict)
0019     routes: dict[str, pd.DataFrame] = field(default_factory=dict)
0020     arrivals: dict[str, pd.DataFrame] = field(default_factory=dict)
0021     customers: pd.DataFrame = field(default_factory=pd.DataFrame)
0022     reuse_schedule: pd.DataFrame = field(default_factory=pd.DataFrame)
0023     sensitivity: pd.DataFrame = field(default_factory=pd.DataFrame)
0024     history: dict[str, Any] = field(default_factory=dict)
0025     geojson: dict[str, Any] = field(default_factory=dict)
0026     manifest: dict[str, Any] = field(default_factory=dict)
0027     errors: list[str] = field(default_factory=list)
0028
0029
0030 def _json(name: str, errors: list[str]) -> dict[str, Any]:
0031     path = DATA_DIR / name
0032     if not path.exists():
0033         errors.append(f"缺少数据文件: {name}")
0034         return {}
0035     try:
0036         return json.loads(path.read_text(encoding="utf-8"))
0037     except Exception as exc:
0038         errors.append(f"无法读取 {name}: {exc}")
0039         return {}
0040
0041
0042 def _csv(name: str, errors: list[str]) -> pd.DataFrame:
0043     path = DATA_DIR / name
0044     if not path.exists():
0045         errors.append(f"缺少数据文件: {name}")
0046         return pd.DataFrame()
0047     try:
0048         return pd.read_csv(path, encoding="utf-8-sig")
0049     except Exception as exc:
0050         errors.append(f"无法读取 {name}: {exc}")
0051         return pd.DataFrame()
0052
0053
0054 @st.cache_data(show_spinner=False)
0055 def load_site_data() -> SiteData:
0056     errors: list[str] = []
0057     summary = _json("summaries.json", errors)
0058     history = _json("algorithm_history.json", errors)
0059     manifest = _json("manifest.json", errors)
0060     geojson = {
0061         "routes": _json("route_features.geojson", errors),
0062         "customers": _json("customer_points.geojson", errors),
0063     }
0064     routes = {
0065         "noodle": _csv("routes_noodle.csv", errors),
0066         "ginger": _csv("routes_ginger.csv", errors),
0067     }
0068     arrivals = {
0069         "noodle": _csv("arrivals_noodle.csv", errors),
0070         "ginger": _csv("arrivals_ginger.csv", errors),
0071     }
0072     return SiteData(
0073         summary=summary,
0074         routes=routes,
0075         arrivals=arrivals,
0076         customers=_csv("customers.csv", errors),
0077         reuse_schedule=_csv("reuse_schedule.csv", errors),
0078         sensitivity=_csv("sensitivity.csv", errors),
0079         history=history,
0080         geojson=geojson,
0081         manifest=manifest,
0082         errors=errors,
0083     )
0084
```

```
0085
0086 def get_summary(data: SiteData, key: str) -> dict[str, Any]:
0087     return data.summary.get(key, {}) or {}
0088
0089
0090 def safe_columns(frame: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
0091     existing = [column for column in columns if column in frame.columns]
0092     return frame[existing].copy()
```

代码附录：功能组件_页面共用代码/order_state.py

用途：订单状态。当前版本共 174 行。

```
0001 from __future__ import annotations
0002
0003 from datetime import datetime
0004 import json
0005 import os
0006 from pathlib import Path
0007 from typing import Any
0008 from urllib.parse import urlencode
0009 from urllib.request import Request, urlopen
0010 from uuid import uuid4
0011
0012 import pandas as pd
0013 import streamlit as st
0014
0015 STATUS_FLOW = ["订单已提交", "仓库备货中", "等待装车", "配送途中", "配送完成"]
0016 ROOT = Path(__file__).resolve().parents[1]
0017 ORDER_DATA_DIR = ROOT / "客户订单数据"
0018 SETTINGS_PATH = ORDER_DATA_DIR / "调价参数.json"
0019 DEPOT_LON, DEPOT_LAT = 104.26104981303031, 30.851137170192068
0020
0021
0022 def staff_password() -> str | None:
0023     """密码只从运行环境或 Streamlit secrets 读取。"""
0024     configured = os.getenv("YL_STAFF_PASSWORD")
0025     if configured:
0026         return configured
0027     try:
0028         return st.secrets.get("YL_STAFF_PASSWORD")
0029     except (FileNotFoundError, RuntimeError):
0030         return None
0031
0032
0033 def pricing_settings() -> dict[str, float]:
0034     defaults = {"起步价_元": 12.0, "里程价_元每km": 0.8, "鲜面条_元每kg": 0.75, "姜蒜_元每kg": 0.41}
0035     try:
0036         saved = json.loads(SETTINGS_PATH.read_text(encoding="utf-8"))
0037         return {key: float(saved.get(key, value)) for key, value in defaults.items()}
0038     except (OSError, ValueError, TypeError):
0039         return defaults
0040
0041
0042 def save_pricing_settings(values: dict[str, float]) -> None:
0043     ORDER_DATA_DIR.mkdir(parents=True, exist_ok=True)
0044     SETTINGS_PATH.write_text(json.dumps(values, ensure_ascii=False, indent=2), encoding="utf-8")
0045
0046
0047 def haversine_km(lon1: float, lat1: float, lon2: float, lat2: float) -> float:
0048     from math import asin, cos, radians, sin, sqrt
0049     dlat, dlon = radians(lat2 - lat1), radians(lon2 - lon1)
0050     a = sin(dlat / 2) ** 2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon / 2) ** 2
0051     return 6371.0088 * 2 * asin(sqrt(a))
0052
0053
0054 def estimate_delivery_fee(product: str, quantity_kg: float, longitude: float | None = None, latitude: float | None = None) -> dict[str, float]:
0055     """报价由起步价、品类重量价与参考里程价组成。"""
0056     rates = pricing_settings()
0057     weight = max(0.0, float(quantity_kg)) * rates[f"{product}_元每kg"]
0058     distance = haversine_km(DEPOT_LON, DEPOT_LAT, longitude, latitude) if longitude is not None and latitude is not None else 0.0
0059     mileage = distance * rates["里程价_元每km"]
0060     return {"起步价_元": rates["起步价_元"], "重量价_元": round(weight, 2), "参考距离_km": round(distance, 2), "里程价_元": round(mileage, 2), "预估费用_元": round(rates["起步价_元"] + weight + mileage, 2)}
0061
0062
0063 @st.cache_data(ttl=86400, show_spinner=False)
0064 def geocode_address(address: str) -> tuple[float, float, str] | None:
0065     """仅在客户主动点击识别时访问外部服务，且缓存 24 小时。"""
0066     query = address.strip()
0067     if len(query) < 6:
0068         return None
0069     url = "https://nominatim.openstreetmap.org/search?" + urlencode({"q": query + " 成都", "format": "jsonv2", "limit": 1, "countrycodes": "cn"})
0070     try:
0071         request = Request(url, headers={"User-Agent": "YinliDeliveryContest/1.0"})
0072         with urlopen(request, timeout=6) as response:
0073             result = json.loads(response.read().decode("utf-8"))
0074             if result:
0075                 return float(result[0]["lon"]), float(result[0]["lat"]), str(result[0].get("display_name", query))
0076     except Exception:
0077         return None
0078     return None
0079
0080
0081 def _route_geojson(longitude: float | None, latitude: float | None) -> dict[str, Any] | None:
0082     if longitude is None or latitude is None:
0083         return None
```

```

0084     return {"type": "FeatureCollection", "features": [{"type": "Feature", "properties": {"label": "订单演示调度线", "route_mode": "演示坐标连线"}, "geometry": {"type": "
LineString", "coordinates": [[DEPOT_LON, DEPOT_LAT], [float(longitude), float(latitude)]]]}}]
0085
0086
0087 def _excel_value(value: Any) -> Any:
0088     return json.dumps(value, ensure_ascii=False) if isinstance(value, (dict, list)) else value
0089
0090
0091 def save_order_to_excel(order: dict[str, Any]) -> str:
0092     delivery_date = str(order.get("期望送达日期") or datetime.now().date())
0093     product = str(order.get("品类", "未分类"))
0094     folder = ORDER_DATA_DIR / delivery_date
0095     folder.mkdir(parents=True, exist_ok=True)
0096     path = folder / f"{product}订单.xlsx"
0097     row = pd.DataFrame([{"key": _excel_value(value) for key, value in order.items() if key not in ("状态序号", "保存路径", "保存状态")}]
0098     existing = pd.read_excel(path) if path.exists() else pd.DataFrame()
0099     pd.concat([existing, row], ignore_index=True).to_excel(path, index=False)
0100     return str(path.relative_to(ROOT))
0101
0102
0103 def _parse_record(item: dict[str, Any], path: Path) -> dict[str, Any]:
0104     for key in ("路线GeoJSON", "异常反馈", "报价明细"):
0105         if isinstance(item.get(key), str) and item[key].strip().startswith(("{", "[")):
0106             try:
0107                 item[key] = json.loads(item[key])
0108             except ValueError:
0109                 pass
0110     item["保存路径"] = str(path.relative_to(ROOT))
0111     item["状态序号"] = STATUS_FLOW.index(item.get("状态")) if item.get("状态") in STATUS_FLOW else 0
0112     item.setdefault("数据模式", "本地演示订单")
0113     return item
0114
0115
0116 def init_orders(include_saved: bool = False) -> list[dict[str, Any]]:
0117     """客户只看当前会话订单；工作人员可读取本机演示订单表。"""
0118     if "customer_orders" not in st.session_state:
0119         st.session_state["customer_orders"] = []
0120     own = st.session_state["customer_orders"]
0121     if not include_saved:
0122         return own
0123     records = {str(item.get("订单编号")): item for item in own}
0124     for path in ORDER_DATA_DIR.glob("/*.xlsx"):
0125         try:
0126             for raw in pd.read_excel(path).fillna("").to_dict("records"):
0127                 item = _parse_record(raw, path)
0128                 records[str(item.get("订单编号"))] = item
0129         except Exception:
0130             continue
0131     return sorted(records.values(), key=lambda item: str(item.get("提交时间", "")), reverse=True)
0132
0133
0134 def create_order(payload: dict[str, Any]) -> dict[str, Any]:
0135     own_orders = init_orders()
0136     now = datetime.now()
0137     longitude = float(payload.get("经度")) if payload.get("经度") not in (None, "") else None
0138     latitude = float(payload.get("纬度")) if payload.get("纬度") not in (None, "") else None
0139     quote = estimate_delivery_fee(str(payload.get("品类", "")), float(payload.get("配送重量_kg", 0)), longitude, latitude)
0140     order = {"订单编号": f"YL{now:%Y%m%d%H%M%S}{uuid4().hex[:4].upper()}", "提交时间": now.strftime("%Y-%m-%d %H:%M:%S"), "状态": STATUS_FLOW[1] if longitude
is not None else STATUS_FLOW[0], "状态序号": 1 if longitude is not None else 0, "数据模式": "本会话演示订单", "报价明细": quote, "预估费用_元": quote["预估费用_元"], "路线
GeoJSON": _route_geojson(longitude, latitude), **payload}
0141     try:
0142         order["保存路径"] = save_order_to_excel(order)
0143         order["保存状态"] = "已保存到本机演示订单表"
0144     except Exception as exc:
0145         order["保存状态"] = f"本机保存失败: {exc}"
0146     own_orders.insert(0, order)
0147     return order
0148
0149
0150 def find_order(order_id: str, include_saved: bool = False) -> dict[str, Any] | None:
0151     return next((item for item in init_orders(include_saved) if item.get("订单编号") == order_id), None)
0152
0153
0154 def persist_order(order: dict[str, Any]) -> None:
0155     path_text = order.get("保存路径")
0156     if not path_text:
0157         return
0158     try:
0159         path = ROOT / str(path_text)
0160         frame = pd.read_excel(path)
0161         mask = frame["订单编号"].astype(str) == str(order["订单编号"])
0162         if mask.any():
0163             for key, value in order.items():
0164                 if key not in ("状态序号", "保存路径", "保存状态"):
0165                     frame.loc[mask, key] = _excel_value(value)
0166             frame.to_excel(path, index=False)
0167     except Exception:
0168         pass
0169
0170

```

```
0171 def advance_order(order: dict[str, Any]) -> None:
0172     index = min(int(order.get("状态序号", 0)) + 1, len(STATUS_FLOW) - 1)
0173     order["状态序号"], order["状态"] = index, STATUS_FLOW[index]
0174     persist_order(order)
```

代码附录：功能组件_页面共用代码/maps.py

用途：地图。当前版本共 121 行。

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005 from typing import Any
0006
0007 import folium
0008
0009
0010 ROOT = Path(__file__).resolve().parents[1]
0011 DATA_DIR = ROOT / "网站数据_页面读取的指标"
0012 DEPOT = [30.851137170192068, 104.26104981303031]
0013
0014
0015 def load_geojson(name: str) -> dict[str, Any]:
0016     path = DATA_DIR / name
0017     if not path.exists():
0018         return {"type": "FeatureCollection", "features": []}
0019     return json.loads(path.read_text(encoding="utf-8"))
0020
0021
0022 def create_chengdu_map(zoom_start: int = 10) -> tuple[folium.Map, folium.GeoJson | None]:
0023     fmap = folium.Map(location=DEPOT, zoom_start=zoom_start, tiles=None, control_scale=True, prefer_canvas=True)
0024     # 只绘制随项目交付的道路 GeoJSON，断网时地图仍可运行。
0025
0026     major_data = load_geojson("road_major.geojson")
0027     minor_data = load_geojson("road_minor.geojson")
0028     if major_data.get("features"):
0029         folium.GeoJson(
0030             major_data,
0031             name="主干道",
0032             style_function=lambda _: {"color": "#6f8190", "weight": 1.7, "opacity": 0.58},
0033             smooth_factor=1.5,
0034             show=True,
0035         ).add_to(fmap)
0036
0037     minor_layer = None
0038     if minor_data.get("features"):
0039         minor_layer = folium.GeoJson(
0040             minor_data,
0041             name="细支道路 (放大显示)",
0042             style_function=lambda _: {"color": "#aeb9c0", "weight": 0.85, "opacity": 0.48},
0043             smooth_factor=2,
0044             show=False,
0045         ).add_to(fmap)
0046
0047     folium.Marker(
0048         DEPOT,
0049         tooltip="银犁配送中心",
0050         icon=folium.Icon(color="darkgreen", icon="home"),
0051     ).add_to(fmap)
0052     return fmap, minor_layer
0053
0054
0055 def add_zoom_detail_behavior(fmap: folium.Map, minor_layer: folium.GeoJson | None, threshold: int = 13) -> None:
0056     if minor_layer is None:
0057         return
0058     map_name = fmap.get_name()
0059     minor_name = minor_layer.get_name()
0060     script = f"""
0061     <script>
0062     document.addEventListener('DOMContentLoaded', function() {{
0063     const map = {map_name};
0064     const minor = {minor_name};
0065     const syncRoadDetail = function() {{
0066     if (map.getZoom() >= {threshold}) {{
0067     if (!map.hasLayer(minor)) minor.addTo(map);
0068     }} else if (map.hasLayer(minor)) {{
0069     map.removeLayer(minor);
0070     }}
0071     }};
0072     map.on('zoomend', syncRoadDetail);
0073     syncRoadDetail();
0074     }});
0075     </script>
0076     """
0077     fmap.get_root().html.add_child(folium.Element(script))
0078
0079
0080 def add_route_features(fmap: folium.Map, geojson: dict[str, Any], product_code: str | None = None) -> None:
0081     colors = {"N": "#1750df", "G": "#ff8133"}
0082     labels = {"N": "鲜面专线", "G": "姜蒜专线"}
0083     groups: dict[str, folium.FeatureGroup] = {}
0084     for code in ("N", "G"):
```

```

0085     groups[code] = folium.FeatureGroup(name=labels[code], show=product_code in (None, code))
0086     groups[code].add_to(fmap)
0087 for feature in geojson.get("features", []):
0088     props = feature.get("properties", {})
0089     code = str(props.get("product_code", "N"))
0090     if product_code and code != product_code:
0091         continue
0092     geometry = feature.get("geometry", {})
0093     coords = geometry.get("coordinates", [])
0094     if geometry.get("type") != "LineString" or not coords or code not in groups:
0095         continue
0096     folium.PolyLine(
0097         [[point[1], point[0]] for point in coords],
0098         color=colors[code],
0099         weight=4,
0100         opacity=0.86,
0101         tooltip=props.get("label", labels[code]),
0102     ).add_to(groups[code])
0103
0104
0105 def add_customer_points(fmap: folium.Map, geojson: dict[str, Any], product_code: str | None = None) -> None:
0106     colors = {"N": "#1750df", "G": "#ff8133"}
0107     for feature in geojson.get("features", []):
0108         props = feature.get("properties", {})
0109         code = str(props.get("product_code", "N"))
0110         if product_code and code != product_code:
0111             continue
0112         coords = feature.get("geometry", {}).get("coordinates", [])
0113         if len(coords) < 2:
0114             continue
0115         popup = "<b>{}</b><br>品类: {}<br>配送量: {} kg<br>时间要求: {}".format(
0116             props.get("label", "客户"), props.get("product", "暂无"), props.get("demand_kg", "暂无"), props.get("time_window", "暂无")
0117         )
0118         folium.CircleMarker(
0119             [coords[1], coords[0]], radius=4.5, color=colors.get(code, "#1750df"),
0120             fill=True, fill_opacity=0.88, popup=popup, tooltip=props.get("label", "客户")
0121         ).add_to(fmap)

```

代码附录：功能组件_页面共用代码/planner.py

用途：快速规划。当前版本共 326 行。

```
0001 from __future__ import annotations
0002
0003 import io
0004 import json
0005 import math
0006 from typing import Any
0007
0008 import pandas as pd
0009
0010
0011 def _number(value: Any, default: float = 0.0) -> float:
0012     try:
0013         return float(value)
0014     except (TypeError, ValueError):
0015         return default
0016
0017
0018 def _minutes(value: Any, default: int = 0) -> int:
0019     if value is None or (isinstance(value, float) and math.isnan(value)):
0020         return default
0021     if isinstance(value, str):
0022         value = value.strip()
0023         if ":" in value:
0024             hh, mm = value.split(":", 1)
0025             return int(hh) * 60 + int(float(mm))
0026         if "-" in value:
0027             value = value.split("-", 1)[0]
0028         return int(float(value))
0029
0030
0031 def _window(value: Any, fallback_start: Any = None, fallback_end: Any = None) -> tuple[int, int]:
0032     if isinstance(value, str) and "-" in value:
0033         left, right = value.split("-", 1)
0034         return _minutes(left), _minutes(right)
0035     return _minutes(fallback_start, 240), _minutes(fallback_end, 420)
0036
0037
0038 def read_uploaded_file(uploaded_file: Any) -> tuple[str, Any]:
0039     """读取上传文件，返回文件名和 DataFrame 或 JSON 对象。"""
0040     name = uploaded_file.name
0041     suffix = name.lower().rsplit(".", 1)[-1] if "." in name else ""
0042     content = uploaded_file.getvalue()
0043     if suffix == "json":
0044         return name, json.loads(content.decode("utf-8-sig"))
0045     if suffix in {"xlsx", "xls"}:
0046         sheets = pd.read_excel(io.BytesIO(content), sheet_name=None, header=None)
0047         # 多工作表成果包优先选取包含客户、经度、纬度的客户数据表。
0048         ranked = []
0049         for sheet_name, sheet in sheets.items():
0050             text = " ".join(str(x) for x in sheet.iloc[12:].to_numpy().ravel() if pd.notna(x))
0051             score = sum(token in text for token in ("客户", "经度", "纬度", "需求"))
0052             ranked.append((score, sheet_name, sheet))
0053         _, selected = max(ranked, key=lambda item: item[0])
0054         return name, selected
0055     try:
0056         return name, pd.read_csv(io.BytesIO(content), encoding="utf-8-sig")
0057     except UnicodeDecodeError:
0058         return name, pd.read_csv(io.BytesIO(content), encoding="gb18030")
0059
0060
0061 def is_result_json(value: Any) -> bool:
0062     return isinstance(value, dict) and isinstance(value.get("summary"), dict) and ("routes" in value or "trips" in value)
0063
0064
0065 def _find_header_row(raw: pd.DataFrame) -> int | None:
0066     for index in range(min(len(raw), 8)):
0067         values = " ".join(str(x) for x in raw.iloc[index].tolist() if pd.notna(x))
0068         if "客户" in values and ("经度" in values or "坐标" in values):
0069             return index
0070     return None
0071
0072
0073 def normalize_customers(raw: pd.DataFrame, product: str) -> pd.DataFrame:
0074     """兼容竞赛模板和普通 CSV，归一化客户需求字段。"""
0075     frame = raw.copy()
0076
0077     # 兼容成果包中的“客户编号横向展开”工作簿：客户编号在列上，字段在行上。
0078     for row_index in range(min(len(frame), 10)):
0079         row_values = frame.iloc[row_index].tolist()
0080         id_position = next((index for index, value in enumerate(row_values) if str(value).strip() in {"客户编号", "客户"}), None)
0081         if id_position is None or len(row_values) - id_position < 4:
0082             continue
0083         value_position = next((index for index in range(id_position + 1, len(row_values)) if pd.notna(pd.to_numeric(row_values[index], errors="coerce"))), None)
0084         if value_position is None:
```

```

0085         continue
0086     ids = pd.to_numeric(pd.Series(row_values[value_position:]), errors="coerce")
0087     if ids.notna().sum() < 3:
0088         continue
0089     labels = frame.iloc[row_index : row_index + 16].apply(lambda row: " ".join(str(x) for x in row.iloc[value_position].tolist() if pd.notna(x)), axis=1)
0090
0091     def row_for(*terms: str) -> pd.Series | None:
0092         hit = labels[labels.apply(lambda text: any(term in text for term in terms))]
0093         return frame.iloc[hit.index[0]] if not hit.empty else None
0094
0095     longitude_row = row_for("经度")
0096     latitude_row = row_for("纬度")
0097     demand_row = row_for("需求量", "日需求", "鲜面条需求")
0098     if longitude_row is None or latitude_row is None or demand_row is None:
0099         continue
0100     values = ("客户编号": ids.to_numpy(), "经度": pd.to_numeric(pd.Series(longitude_row.iloc[value_position:]), errors="coerce").to_numpy(), "纬度":
pd.to_numeric(pd.Series(latitude_row.iloc[value_position:]), errors="coerce").to_numpy())
0101     if product == "大蒜":
0102         garlic_row = row_for("蒜需求", "大蒜")
0103         ginger_row = row_for("姜需求", "生姜")
0104         if garlic_row is not None and ginger_row is not None:
0105             values["大蒜_kg"] = pd.to_numeric(pd.Series(garlic_row.iloc[value_position:]), errors="coerce").to_numpy()
0106             values["生姜_kg"] = pd.to_numeric(pd.Series(ginger_row.iloc[value_position:]), errors="coerce").to_numpy()
0107         else:
0108             values["需求量_kg"] = pd.to_numeric(pd.Series(demand_row.iloc[value_position:]), errors="coerce").to_numpy()
0109     else:
0110         values["需求量_kg"] = pd.to_numeric(pd.Series(demand_row.iloc[value_position:]), errors="coerce").to_numpy()
0111     for target, terms in {"时间窗": ("期望时间窗", "期望配送时间窗"), "可接受时间窗": ("可接受时间窗",), "服务时间_分": ("服务时间",)}.items():
0112         matched = row_for(*terms)
0113         if matched is not None:
0114             values[target] = matched.iloc[value_position:].to_numpy()
0115     frame = pd.DataFrame(values)
0116     break
0117
0118 header_row = _find_header_row(frame)
0119 if header_row is not None and header_row >= 0:
0120     frame = frame.iloc[header_row + 1:].copy()
0121     # 竞赛工作簿的字段占两行: 第一行是业务字段, 下一行是“经度/纬度”等子字段。
0122     # 该子标题不是客户记录, 必须在归一化前删除。
0123     if not frame.empty:
0124         first_row = frame.iloc[0].astype(str).str.cat(sep=" ")
0125         first_id = pd.to_numeric(frame.iloc[0, 0], errors="coerce")
0126         if pd.isna(first_id) and ("经度" in first_row or "纬度" in first_row or "时间窗" in first_row):
0127             frame = frame.iloc[1:].copy()
0128     if frame.shape[1] >= 3 and not any("客户" in str(c) for c in frame.columns):
0129         if product == "鲜面条" and frame.shape[1] >= 9:
0130             frame = frame.iloc[:, :9]
0131             frame.columns = ["客户编号", "经度", "纬度", "需求量_kg", "期望开始", "期望结束", "可接受开始", "可接受结束", "服务时间_分"]
0132         elif product == "大蒜" and frame.shape[1] >= 10:
0133             frame = frame.iloc[:, :10]
0134             frame.columns = ["客户编号", "经度", "纬度", "大蒜_kg", "生姜_kg", "期望开始", "期望结束", "可接受开始", "可接受结束", "服务时间_分"]
0135     aliases = {
0136         "客户编号": ["客户编号", "客户", "customer", "id", "编号"],
0137         "经度": ["经度", "longitude", "lon", "x"],
0138         "纬度": ["纬度", "latitude", "lat", "y"],
0139         "需求量_kg": ["需求量_kg", "需求量", "需求量(kg)", "demand", "demand_kg"],
0140         "期望开始": ["期望开始", "期望时间窗开始", "期望窗开始", "earliest"],
0141         "期望结束": ["期望结束", "期望时间窗结束", "期望窗结束", "latest"],
0142         "可接受开始": ["可接受开始", "可接受时间窗开始", "acceptable_start"],
0143         "可接受结束": ["可接受结束", "可接受时间窗结束", "acceptable_end"],
0144         "服务时间_分": ["服务时间_分", "服务时间", "service_min", "service_time"],
0145         "时间窗": ["时间窗", "期望时间窗", "time_window", "window"],
0146     }
0147     renamed: dict[str, str] = {}
0148     for target, candidates in aliases.items():
0149         for column in frame.columns:
0150             compact = str(column).replace(" ", "").replace("\n", "")
0151             if compact in (str(candidate).replace(" ", "") for candidate in candidates):
0152                 renamed[column] = target
0153             break
0154     frame = frame.rename(columns=renamed)
0155     if product == "大蒜" and "需求量_kg" not in frame.columns and {"大蒜_kg", "生姜_kg"}.issubset(frame.columns):
0156         frame["需求量_kg"] = pd.to_numeric(frame["大蒜_kg"], errors="coerce").fillna(0) + pd.to_numeric(frame["生姜_kg"], errors="coerce").fillna(0)
0157     required = ["客户编号", "经度", "纬度", "需求量_kg"]
0158     missing = [column for column in required if column not in frame.columns]
0159     if missing:
0160         raise ValueError("缺少必要字段: " + " ".join(missing) + "。需要客户编号、经度、纬度和需求量。")
0161     for column in ["客户编号", "经度", "纬度", "需求量_kg", "服务时间_分", "期望开始", "期望结束", "可接受开始", "可接受结束"]:
0162         if column in frame.columns:
0163             frame[column] = pd.to_numeric(frame[column], errors="coerce")
0164     frame = frame[frame["客户编号"].notna() & frame["经度"].notna() & frame["纬度"].notna() & frame["需求量_kg"].notna()].copy()
0165     if frame.empty:
0166         raise ValueError("没有读取到有效客户记录。")
0167     if "时间窗" not in frame.columns:
0168         frame["时间窗"] = frame.apply(lambda row: f"{_minutes(row.get('期望开始'), 240):03d}-{_minutes(row.get('期望结束'), 420):03d}", axis=1)
0169     frame["产品"] = product
0170     frame["服务时间_分"] = frame.get("服务时间_分", pd.Series(15, index=frame.index)).fillna(15)
0171     frame["客户编号"] = frame["客户编号"].astype(int)
0172     return frame.reset_index(drop=True)
0173

```

```

0174
0175 def haversine_km(lon1: float, lat1: float, lon2: float, lat2: float) -> float:
0176     radius = 6371.0088
0177     phi1, phi2 = math.radians(lat1), math.radians(lat2)
0178     dphi = math.radians(lat2 - lat1)
0179     dlambda = math.radians(lon2 - lon1)
0180     value = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlambda / 2) ** 2
0181     return radius * 2 * math.atan2(math.sqrt(value), math.sqrt(1 - value)) * 1.15
0182
0183
0184 def generate_plan(customers: pd.DataFrame, settings: dict[str, float]) -> dict[str, Any]:
0185     """基于客户坐标的快速可行初算，不替代正式最短路和 ALNS 求解。"""
0186     depot_lon = settings.get("depot_lon", 104.26105)
0187     depot_lat = settings.get("depot_lat", 30.851137)
0188     capacity = max(1.0, settings.get("capacity_kg", 1500.0))
0189     requested_vehicles = max(1, int(settings.get("vehicles", 1)))
0190     required_by_capacity = math.ceil(float(customers["需求量_kg"].sum()) / capacity)
0191     vehicle_count = max(requested_vehicles, required_by_capacity)
0192     speed = max(1.0, settings.get("speed_kmh", 35.0))
0193     fixed_rate = settings.get("fixed_cost", 200.0)
0194     transport_rate = settings.get("transport_cost_per_km", 2.5)
0195     cooling_rate = settings.get("cooling_cost_per_km", 0.45)
0196     late_rate = settings.get("late_penalty_per_min", 0.5)
0197     loss_rate = settings.get("loss_cost_per_kg", 0.0336)
0198     remaining = {int(row["客户编号"]): row for _, row in customers.iterrows()}
0199     routes: list[list[pd.Series]] = []
0200     for _ in range(vehicle_count):
0201         route: list[pd.Series] = []
0202         load = 0.0
0203         current_lon, current_lat = depot_lon, depot_lat
0204         while remaining:
0205             candidates = []
0206             for customer_id, row in remaining.items():
0207                 demand = float(row["需求量_kg"])
0208                 if load + demand <= capacity + 1e-9:
0209                     start, _ = _window(row.get("时间窗"), row.get("期望开始"), row.get("期望结束"))
0210                     distance = haversine_km(current_lon, current_lat, float(row["经度"]), float(row["纬度"]))
0211                     candidates.append((distance + abs(start - 300) * 0.002, customer_id, row))
0212             if not candidates:
0213                 break
0214             _, customer_id, row = min(candidates, key=lambda item: item[0])
0215             route.append(row)
0216             load += float(row["需求量_kg"])
0217             current_lon, current_lat = float(row["经度"]), float(row["纬度"])
0218             del remaining[customer_id]
0219         if route:
0220             routes.append(route)
0221     while remaining:
0222         route = []
0223         load = 0.0
0224         current_lon, current_lat = depot_lon, depot_lat
0225         while remaining:
0226             candidates = [(haversine_km(current_lon, current_lat, float(row["经度"]), float(row["纬度"])), customer_id, row) for customer_id, row in remaining.items() if load +
float(row["需求量_kg"]) <= capacity + 1e-9]
0227             if not candidates:
0228                 raise ValueError("存在需求量超过车辆额定载重的客户，无法生成容量可行方案。")
0229             _, customer_id, row = min(candidates, key=lambda item: item[0])
0230             route.append(row)
0231             load += float(row["需求量_kg"])
0232             current_lon, current_lat = float(row["经度"]), float(row["纬度"])
0233             del remaining[customer_id]
0234             if len(route) >= 8:
0235                 break
0236             routes.append(route)
0237     route_rows, arrival_rows = [], []
0238     total_distance = total_fixed = total_transport = total_cooling = total_penalty = total_loss = 0.0
0239     ontime = 0
0240     for vehicle_index, route in enumerate(routes, 1):
0241         if not route:
0242             continue
0243         earliest = min(_window(row.get("时间窗"), row.get("期望开始"), row.get("期望结束"))[0] for row in route)
0244         clock = max(0.0, earliest - 60)
0245         current_lon, current_lat = depot_lon, depot_lat
0246         load = sum(float(row["需求量_kg"]) for row in route)
0247         vehicle_distance = 0.0
0248         vehicle_penalty = 0.0
0249         vehicle_ontime = 0
0250         for stop_index, row in enumerate(route, 1):
0251             leg = haversine_km(current_lon, current_lat, float(row["经度"]), float(row["纬度"]))
0252             travel = leg / speed * 60
0253             arrival = clock + travel
0254             start, end = _window(row.get("时间窗"), row.get("期望开始"), row.get("期望结束"))
0255             service_start = max(arrival, start)
0256             service = _number(row.get("服务时间_分"), 15)
0257             late = max(0.0, service_start - end)
0258             early = max(0.0, start - arrival)
0259             is_ontime = arrival >= start and arrival <= end
0260             if is_ontime:
0261                 ontime += 1
0262                 vehicle_ontime += 1

```

```

0263     vehicle_penalty += late * late_rate
0264     arrival_rows.append(("车辆编号": vehicle_index, "停靠序号": stop_index, "客户编号": int(row["客户编号"]), "产品": row["产品"], "需求量_kg": float(row["需求量_kg"]), "路段
距离_km": leg, "路段速度_kmh": speed, "到达时刻": f"{int(arrival // 60):02d}:{int(arrival % 60):02d}", "开始服务时刻": f"{int(service_start // 60):02d}:{int(service_start % 60):02d}", "早到
偏差_min": early, "迟到偏差_min": late, "期望窗内": is_ontime))
0265     clock = service_start + service
0266     current_lon, current_lat = float(row["经度"]), float(row["纬度"])
0267     vehicle_distance += leg
0268     return_leg = haversine_km(current_lon, current_lat, depot_lon, depot_lat)
0269     vehicle_distance += return_leg
0270     clock += return_leg / speed * 60
0271     fixed = fixed_rate
0272     transport = vehicle_distance * transport_rate
0273     cooling = vehicle_distance * cooling_rate
0274     loss = load * loss_rate
0275     total_fixed += fixed
0276     total_transport += transport
0277     total_cooling += cooling
0278     total_penalty += vehicle_penalty
0279     total_loss += loss
0280     total_distance += vehicle_distance
0281     route_rows.append(("车辆编号": vehicle_index, "产品": route[0]["产品"], "车型": "上传参数车辆", "客户数": len(route), "配送量_kg": load, "额定载重_kg": capacity, "装载率":
load / capacity, "发车时刻": f"{int((earliest - 60) // 60):02d}:{int((earliest - 60) % 60):02d}", "回场时刻": f"{int(clock // 60):02d}:{int(clock % 60):02d}", "总里程_km": vehicle_distance, "
固定成本_元": fixed, "运输成本_元": transport, "制冷成本_元": cooling, "时间窗惩罚_元": vehicle_penalty, "货损成本_元": loss, "总成本_元": fixed + transport + cooling +
vehicle_penalty + loss, "访问顺序": "配送中心0→" + "→".join(f"客户{int(row['客户编号']):02d}" for row in route) + "→配送中心0", "准时客户": vehicle_ontime))
0282     total_cost = total_fixed + total_transport + total_cooling + total_penalty + total_loss
0283     summary = {"solution_type": "上传数据快速可行初算, 不替代正式 Dijkstra 与 ALNS 求解", "customers": len(customers), "vehicles_used": len(route_rows), "total_load_kg":
float(customers["需求量_kg"].sum()), "total_distance_km": total_distance, "fixed_cost": total_fixed, "transport_cost": total_transport, "cooling_cost": total_cooling, "
time_penalty_cost": total_penalty, "loss_cost": total_loss, "total_cost": total_cost, "ontime_customers": ontime, "ontime_rate": ontime / len(customers), "average_load_rate":
sum(row["装载率"] for row in route_rows) / len(route_rows), "cost_breakdown": {"fixed_cost": total_fixed, "transport_cost": total_transport, "cooling_cost": total_cooling, "
time_penalty_cost": total_penalty, "loss_cost": total_loss}, "validations": {"all_customers_once": len(arrival_rows) == len(customers), "capacity": all(row["配送量_kg"] <= row["额定
载重_kg"] for row in route_rows), "coordinates_available": True}}
0284     return {"summary": summary, "routes": pd.DataFrame(route_rows), "arrivals": pd.DataFrame(arrival_rows), "customers": customers.copy(), "mode": "快速可行初算"}
0285
0286
0287 def result_json_to_plan(payload: dict[str, Any], product: str) -> dict[str, Any]:
0288     summary = dict(payload.get("summary", {}))
0289     summary.setdefault("solution_type", "已保存最终结果")
0290     route_rows = []
0291     arrival_rows = []
0292     # 第一问/姜蒜使用 routes; 车辆复用结果使用 trips, 统一转换成页面表格。
0293     route_items = payload.get("routes", []) or payload.get("trips", [])
0294     for index, route in enumerate(route_items, 1):
0295         if not isinstance(route, dict):
0296             continue
0297         route_product = route.get("product", product)
0298         vehicle = route.get("vehicle_slot") or route.get("physical_vehicle") or f"趟次{index}"
0299         capacity = route.get("capacity")
0300         load = route.get("load")
0301         route_rows.append(("车辆编号": vehicle, "产品": route_product, "车型": route.get("vehicle_kind") or route.get("vehicle_type"), "客户数": len(route.get("route", [])), "配送量
_kg": load, "额定载重_kg": capacity, "装载率": number(load) / max(number(capacity, 1), 1), "发车时刻": route.get("departure"), "回场时刻": route.get("return_time"), "总里程_km":
route.get("distance"), "固定成本_元": route.get("fixed"), "运输成本_元": route.get("transport"), "制冷成本_元": route.get("cooling"), "时间窗惩罚_元": route.get("time_penalty"), "货损
成本_元": route.get("loss"), "总成本_元": route.get("cost") or route.get("variable_cost"), "物理车辆": route.get("physical_vehicle"), "访问顺序": "配送中心0→" + "→".join(f"客户
{int(x):02d}" for x in route.get("route", [])) + "→配送中心0"))
0302         for detail in route.get("details", []):
0303             if isinstance(detail, dict):
0304                 arrival_rows.append(("车辆编号": vehicle, "产品": route_product, "客户编号": detail.get("customer"), "到达时刻_分钟": detail.get("arrival"), "开始服务时刻_分钟":
detail.get("service_start"), "离开时刻_分钟": detail.get("leave"), "早到偏差_min": detail.get("early"), "迟到偏差_min": detail.get("late"), "期望窗内": detail.get("ontime"), "路段距离
_km": detail.get("leg_distance"), "装载前_kg": detail.get("load_before_leg"))
0305     routes_frame = pd.DataFrame(route_rows)
0306     arrivals_frame = pd.DataFrame(arrival_rows)
0307     summary.setdefault("total_cost", summary.get("cost"))
0308     summary.setdefault("total_distance_km", summary.get("distance"))
0309     summary.setdefault("vehicles_used", summary.get("physical_vehicle_count"))
0310     if summary.get("customers") is None and not arrivals_frame.empty and "客户编号" in arrivals_frame:
0311         identity = [column for column in ["产品", "客户编号"] if column in arrivals_frame]
0312         summary["customers"] = int(arrivals_frame.drop_duplicates(identity).shape[0])
0313     if summary.get("total_load_kg") is None and not routes_frame.empty:
0314         summary["total_load_kg"] = float(pd.to_numeric(routes_frame["配送量_kg"], errors="coerce").sum())
0315     if summary.get("ontime_customers") is None and not arrivals_frame.empty and "期望窗内" in arrivals_frame:
0316         summary["ontime_customers"] = int(arrivals_frame["期望窗内"].fillna(False).astype(bool).sum())
0317     if summary.get("ontime_rate") is None and summary.get("customers"):
0318         summary["ontime_rate"] = _number(summary.get("ontime_customers")) / _number(summary.get("customers"), 1)
0319     cost_fields = ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"]
0320     for summary_key in cost_fields:
0321         if summary.get(summary_key) is None and not routes_frame.empty:
0322             route_key = {"fixed_cost": "固定成本_元", "transport_cost": "运输成本_元", "cooling_cost": "制冷成本_元", "time_penalty_cost": "时间窗惩罚_元", "loss_cost": "货损成本_元"}
            [summary_key]
0323         if route_key in routes_frame:
0324             summary[summary_key] = float(pd.to_numeric(routes_frame[route_key], errors="coerce").sum())
0325     summary["cost_breakdown"] = {key: summary.get(key) for key in cost_fields if summary.get(key) is not None}
0326     return {"summary": summary, "routes": routes_frame, "arrivals": arrivals_frame, "customers": pd.DataFrame(), "mode": "已保存最终结果"}

```

代码附录：功能组件_页面共用代码/matlab_bridge.py

用途：MATLAB 桥。当前版本共 31 行。

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004 import subprocess
0005 from typing import Tuple
0006
0007
0008 ROOT = Path(__file__).resolve().parents[1]
0009 MODEL_DIR = ROOT.parent / "matlab求解第一问"
0010 MATLAB_EXE = Path(r"D:\A-university\RuanJian\MATLAB\bin\matlab.exe")
0011
0012
0013 def run_noodle_model() -> Tuple[bool, str]:
0014     """运行团队已核验的第一问 MATLAB 程序，不替换为页面内快速初算。"""
0015     script = MODEL_DIR / "solve_first_question_noodle.m"
0016     if not MATLAB_EXE.is_file():
0017         return False, "未找到本机 MATLAB，无法运行正式模型。"
0018     if not script.is_file():
0019         return False, "未找到正式鲜面条 MATLAB 求解程序。"
0020     command = f"cd('{str(MODEL_DIR).replace(chr(92), '/')}') solve_first_question_noodle"
0021     try:
0022         result = subprocess.run([str(MATLAB_EXE), "-batch", command], cwd=MODEL_DIR, capture_output=True, text=True, encoding="utf-8", errors="replace",
0023                                 timeout=900)
0024     except subprocess.TimeoutExpired:
0025         return False, "MATLAB 求解超过 15 分钟仍未结束，请在本机 MATLAB 中继续查看。"
0026     if result.returncode != 0:
0027         detail = (result.stderr or result.stdout).strip()[-700:]
0028         return False, f"MATLAB 求解失败：{detail}"
0029     output = MODEL_DIR / "第一问_鲜面条配送优化结果.xlsx"
0030     if not output.is_file():
0031         return False, "MATLAB 已结束，但未找到预期的结果工作簿。"
0032     return True, f"正式 MATLAB 模型已完成，结果保存在：{output}"
```

代码附录：功能组件_页面共用代码/ui.py

用途：统一 UI。当前版本共 303 行。

```
0001 from __future__ import annotations
0002
0003 import html
0004 from typing import Any
0005
0006 import streamlit as st
0007
0008 from 功能组件_页面共用代码.order_state import staff_password
0009
0010
0011 def inject_css() -> None:
0012     st.markdown(
0013         """
0014         <style>
0015         :root { --ink:#1d2a3a; --ink-soft:#415166; --paper:#f5f7fb; --surface:#ffffff; --surface-blue:#eef4ff; --line:#dfe7f1; --muted:#718096; --brand:#1750df;
--brand-dark:#103c9e; --accent:#ff8133; --accent-soft:#fff0e6; --success:#18794e; --success-soft:#e7f6ee; }
0016         .stApp { background:var(--paper); color:var(--ink); font-family:'Microsoft YaHei UI','Microsoft YaHei','Segoe UI',sans-serif; }
0017         [data-testid='stHeader'] { background:rgba(245,247,251,.94); }
0018         [data-testid='stSidebar'] { background:#eef3f9; border-right:1px solid var(--line); }
0019         [data-testid='stSidebar'] * { color:var(--ink); }
0020         [data-testid='stSidebarNav'] { display:none!important; }
0021         html { scroll-behavior:smooth; }
0022         ::selection { background:rgba(255,129,51,.24); color:var(--ink); }
0023         * { scrollbar-width:thin; scrollbar-color:aebbd0 transparent; }
0024         .block-container { max-width:1360px; padding-top:1.35rem; padding-bottom:3.5rem; }
0025         h1,h2,h3,h4 { color:var(--ink); letter-spacing:-.025em; }
0026         h1 { font-size:2.65rem!important; line-height:1.12!important; font-weight:800!important; }
0027         h2 { font-size:1.9rem!important; line-height:1.2!important; }
0028         h3 { font-size:1.2rem!important; margin-top:1.6rem!important; }
0029         p,li,label,.stCaption { color:var(--ink-soft); line-height:1.65; }
0030         .brand-mark { width:42px; height:42px; display:grid; place-items:center; background:var(--brand); color:#fff!important; font-weight:800; letter-spacing:.05em;
border-radius:12px; margin-bottom:8px; box-shadow:0 6px 16px rgba(23,80,223,.18); }
0031         .brand-name { font-weight:800; font-size:1.15rem; }
0032         .nav-section-label { margin:.2rem 0 .55rem; color:var(--muted); font-size:.72rem; font-weight:800; letter-spacing:.08em; }
0033         .page-kicker,.hero-kicker { color:var(--brand); font-size:.72rem; font-weight:800; letter-spacing:.12em; margin-bottom:.45rem; }
0034         .page-subtitle { color:var(--muted); font-size:1rem; margin-top:-.4rem; margin-bottom:1.45rem; }
0035         .hero-panel { background:linear-gradient(135deg,#123d9e 0%,#1750df 65%,#2868e9 100%); color:#fff; border-radius:16px; padding:2rem 2.2rem; display:grid;
grid-template-columns:1.5fr .8fr; gap:2rem; align-items:end; margin:1.1rem 0 1.4rem; box-shadow:0 14px 30px rgba(16,60,158,.16); }
0036         .hero-panel h2 { color:#fff; margin:.2rem 0 .7rem; }
0037         .hero-panel p { color:#e2ebff; max-width:62ch; margin:0; line-height:1.7; }
0038         .hero-kicker { color:#ffd2b5; }
0039         .hero-index { border-left:1px solid rgba(255,255,255,.25); padding-left:1.5rem; display:grid; gap:.35rem; }
0040         .hero-index span,.hero-index small { color:#c9d9ff; font-size:.76rem; letter-spacing:.06em; }
0041         .hero-index strong { color:#fff; font-size:1.3rem; letter-spacing:.04em; }
0042         .metric-card { background:var(--surface); border:1px solid var(--line); border-radius:12px; padding:1rem 1.1rem; min-height:116px; box-shadow:0 4px 12px
rgba(35,64,105,.04); }
0043         .metric-value { font-size:1.85rem; font-weight:800; color:var(--brand-dark); line-height:1.15; }
0044         .metric-label { font-size:.95rem; font-weight:700; margin-top:.55rem; }
0045         .metric-note { color:var(--muted); font-size:.78rem; margin-top:.2rem; }
0046         .route-board,.note-panel,.table-panel { background:var(--surface); border:1px solid var(--line); border-radius:12px; padding:1rem 1.15rem; box-shadow:0 4px 12px
rgba(35,64,105,.04); }
0047         .route-row { display:grid; grid-template-columns:18px 1fr auto; gap:.8rem; align-items:center; padding:.85rem 0; border-bottom:1px solid var(--line); }
0048         .route-row:last-child { border-bottom:0; }
0049         .route-row b { display:block; }
0050         .route-row small { color:var(--muted); display:block; margin-top:.2rem; }
0051         .route-row strong { color:var(--brand); letter-spacing:.04em; }
0052         .route-dot { width:11px; height:11px; border-radius:50%; display:block; }
0053         .route-dot.noodle { background:var(--brand); } .route-dot.ginger { background:var(--accent); } .route-dot.reuse { background:#748297; }
0054         .note-panel { border-color:#f2cfb7; background:#fffaf6; }
0055         .note-panel p { margin:.2rem 0 .8rem; line-height:1.65; }
0056         .note-panel p:last-child { margin-bottom:.2rem; }
0057         .muted { color:var(--muted)!important; font-size:.9rem; }
0058         .section-label { color:var(--brand); font-weight:800; font-size:.88rem; margin:1.55rem 0 .55rem; }
0059         .section-intro { max-width:68ch; color:var(--ink-soft); margin:-.15rem 0 1rem; }
0060         [data-testid='stPageLink-NavLink'] { min-height:2.65rem; display:flex; align-items:center; padding:.75rem 1rem; border:1px solid var(--line)!important;
border-radius:12px; background:var(--surface)!important; color:var(--brand-dark)!important; box-shadow:0 4px 12px rgba(35,64,105,.04); }
0061         [data-testid='stPageLink-NavLink']:hover { border-color:var(--brand)!important; background:var(--surface-blue)!important; color:var(--brand-dark)!important;
transform:translateY(-2px); box-shadow:0 8px 18px rgba(23,80,223,.12); }
0062         [data-testid='stPageLink-NavLink']:focus-visible { outline:3px solid rgba(255,129,51,.38); outline-offset:2px; }
0063         .status-good { color:var(--success); background:var(--success-soft); padding:.24rem .6rem; border-radius:999px; font-size:.82rem; font-weight:700; }
0064         .status-warn { color:#b45309; background:var(--accent-soft); padding:.24rem .6rem; border-radius:999px; font-size:.82rem; font-weight:700; }
0065         .source-note { color:var(--muted); font-size:.78rem; border-top:1px solid var(--line); padding-top:.8rem; margin-top:.2rem; }
0066         .stButton>button,.stDownloadButton>button,[data-testid='stPageLink-NavLink'] { transition:transform .18s cubic-bezier(.16,1,.3,1), box-shadow .18s ease,
border-color .18s ease, background-color .18s ease, color .18s ease; }
0067         .stButton>button { border-radius:9px; border:1px solid var(--brand); color:var(--brand); background:var(--surface); font-weight:700; min-height:2.65rem; }
0068         .stButton>button:hover { border-color:var(--accent); color:var(--accent); background:#fffaf7; transform:translateY(-1px); box-shadow:0 6px 14px rgba(255,129,51,.12);
}
0069         .stButton>button:active,.stDownloadButton>button:active,[data-testid='stPageLink-NavLink']:active { transform:translateY(1px) scale(.98); box-shadow:none; }
0070         .stButton>button[kind='primary'] { background:var(--accent); border-color:var(--accent); color:#fff; box-shadow:0 5px 14px rgba(255,129,51,.2); }
0071         .stButton>button[kind='primary']:hover { background:#e96f24; border-color:#e96f24; color:#fff; }
0072         .stDownloadButton>button { border-radius:9px; background:var(--brand); color:#fff; font-weight:700; border:1px solid var(--brand); }
0073         .stDownloadButton>button:hover { background:var(--brand-dark); border-color:var(--brand-dark); transform:translateY(-1px); box-shadow:0 6px 14px
rgba(23,80,223,.16); }
0074         button:focus-visible,a:focus-visible { outline:3px solid rgba(255,129,51,.42)!important; outline-offset:3px!important; }
```

```

0075 [data-testid='stMetric'] { background:var(--surface); border:1px solid var(--line); padding:.85rem 1rem; border-radius:12px; box-shadow:0 4px 12px
rgba(35,64,105,.04); }
0076 [data-testid='stMetricLabel'] { color:var(--muted); }
0077 [data-testid='stMetricValue'] { color:var(--brand-dark); }
0078 [data-testid='stDataFrame'] { border:1px solid var(--line); border-radius:10px; overflow:hidden; background:var(--surface); }
0079 [data-baseweb='select'] > div, [data-testid='stFileUploader'] section { border-color:var(--line); border-radius:10px; background:var(--surface-blue);
transition:border-color .18s ease, box-shadow .18s ease, background-color .18s ease; }
0080 [data-baseweb='select'] > div:over { border-color:var(--brand); box-shadow:0 4px 12px rgba(23,80,223,.08); }
0081 [data-baseweb='select'] > div:focus-within { border-color:var(--brand); box-shadow:0 0 3px rgba(23,80,223,.14); }
0082 input:focus, textarea:focus { border-color:var(--brand)!important; box-shadow:0 0 2px rgba(23,80,223,.14)!important; }
0083 .home-nav { display:flex; align-items:center; justify-content:space-between; padding:.25rem 0 1rem; border-bottom:1px solid rgba(29,42,58,.09); }
0084 .home-brand { display:flex; align-items:center; gap:.75rem; color:var(--ink); }
0085 .home-brand span { width:34px; height:34px; display:grid; place-items:center; border-radius:10px; background:var(--brand); color:#fff; font-size:.78rem;
font-weight:900; box-shadow:0 8px 22px rgba(23,80,223,.18); }
0086 .home-brand strong { font-size:1rem; letter-spacing:-.01em; }
0087 .home-nav-copy { color:var(--muted); font-size:.84rem; }
0088 .st-key-home_hero { position:relative; overflow:hidden; margin-top:1.25rem; padding:clamp(1.4rem,4vw,4.1rem); border:1px solid rgba(23,80,223,.1);
border-radius:28px; background:linear-gradient(145deg,#fff 0%,#f5f8ff 54%,#eaf1ff 100%); box-shadow:0 22px 70px rgba(24,59,120,.1); isolation:isolate; }
0089 .st-key-home_hero::before { content:''; position:absolute; z-index:-1; width:460px; height:460px; right:-180px; top:-210px; border-radius:50%;
background:radial-gradient(circle,rgba(23,80,223,.18),rgba(23,80,223,0) 68%); animation:home-ambient 8s ease-in-out infinite alternate; }
0090 .home-hero-copy h1 { max-width:11ch; margin:0 0 1.15rem; font-size:clamp(2.75rem,5.3vw,5.7rem)!important; line-height:.99!important; letter-spacing:-.065em;
color:#102c6d; }
0091 .home-hero-copy p { max-width:28rem; margin:0 0 1.35rem; font-size:clamp(1rem,1.4vw,1.2rem); color:#52627a; }
0092 .st-key-home_hero [data-testid='stImage'] { overflow:hidden; border-radius:20px; border:1px solid rgba(23,80,223,.13); background:#fff; box-shadow:0 22px 48px
rgba(23,56,119,.18); animation:home-visual .95s .14s cubic-bezier(.16,1,.3,1) both; }
0093 .st-key-home_hero [data-testid='stImage'] img { transition:transform .8s cubic-bezier(.16,1,.3,1), filter .8s ease; }
0094 .st-key-home_hero [data-testid='stImage']:hover img { transform:scale(1.025); filter:saturate(1.06); }
0095 .st-key-home_hero [data-testid='stImage'] p { padding:.35rem .5rem .5rem; color:var(--muted); }
0096 .st-key-home_hero .home-hero-copy { animation:home-copy .8s cubic-bezier(.16,1,.3,1) both; }
0097 .st-key-home_hero .stButton>button { min-height:3rem; }
0098 .customer-journey { display:grid; gap:.75rem; padding:1.1rem; border:1px solid var(--line); border-radius:16px; background:#fff; box-shadow:0 20px 42px
rgba(31,57,100,.10); }
0099 .customer-journey>div { display:grid; grid-template-columns:34px 1fr; gap:.18rem .8rem; align-items:center; padding:1rem; border-bottom:1px solid var(--line); }
0100 .customer-journey>div:last-child { border-bottom:0; }
0101 .customer-journey span { grid-row:span 2; width:32px; height:32px; display:grid; place-items:center; border-radius:50%; background:var(--surface-blue);
color:var(--brand); font-size:.72rem; font-weight:800; }
0102 .customer-journey b { color:var(--ink); }
0103 .customer-journey small { color:var(--muted); }
0104 .home-proof { display:grid; grid-template-columns:repeat(4,1fr); margin:1rem 0 6rem; padding:1.2rem 1.4rem; border-bottom:1px solid var(--line); }
0105 .home-proof>div { padding:.35rem 1.2rem; border-right:1px solid var(--line); }
0106 .home-proof>div:last-child { border-right:0; }
0107 .home-proof strong { display:block; color:#133c95; font-size:clamp(1.55rem,2.4vw,2.35rem); line-height:1.05; letter-spacing:-.04em; }
0108 .home-proof span { display:block; margin-top:.45rem; color:var(--muted); font-size:.82rem; }
0109 .home-section { max-width:790px; margin:0 0 1.65rem; padding-top:1.4rem; }
0110 .home-section h2 { margin:0 0 .65rem; color:#102c6d; font-size:clamp(2rem,3.8vw,3.75rem)!important; line-height:1.05!important; letter-spacing:-.055em; }
0111 .home-section p { max-width:43rem; margin:0; font-size:1.05rem; color:var(--muted); }
0112 .home-bento { display:grid; grid-template-columns:1.22fr .78fr; grid-template-rows:auto auto; gap:1rem; margin:0 0 6rem; }
0113 .home-feature { position:relative; overflow:hidden; min-height:245px; padding:2rem; border-radius:20px; border:1px solid var(--line); background:#fff;
transition:transform .38s cubic-bezier(.16,1,.3,1), box-shadow .38s ease, border-color .38s ease; }
0114 .home-feature:over { transform:translateY(-6px); border-color:rgba(23,80,223,.24); box-shadow:0 22px 42px rgba(31,57,100,.11); }
0115 .home-feature-large { grid-row:span 2; min-height:510px; background:linear-gradient(150deg,#edf3ff,#dfeaff); }
0116 .home-feature-blue { background:#164bbf; border-color:#164bbf; }
0117 .home-feature-dark { background:#162237; border-color:#162237; }
0118 .home-feature h3 { max-width:13ch; margin:3.5rem 0 .8rem!important; font-size:clamp(1.5rem,2.4vw,2.5rem)!important; line-height:1.08!important; color:#123579; }
0119 .home-feature p { max-width:31rem; margin:0; color:#637089; }
0120 .home-feature-blue h3,home-feature-dark h3 { color:#fff; }
0121 .home-feature-blue p,home-feature-dark p { color:#dce7ff; }
0122 .feature-number { color:var(--accent); font-size:.8rem; font-weight:800; }
0123 .feature-line { position:absolute; left:2rem; right:2rem; bottom:2rem; height:2px; background:linear-gradient(90deg,var(--brand),var(--accent),transparent);
transform-origin:left; animation:line-grow 1.2s .35s cubic-bezier(.16,1,.3,1) both; }
0124 .feature-line::after { content:''; position:absolute; left:58%; top:-5px; width:12px; height:12px; border-radius:50%; background:var(--accent); box-shadow:0 0 0 7px
rgba(255,129,51,.16); }
0125 .st-key-home_flow { margin-bottom:.55rem; padding:2.2rem; border-radius:22px; background:#102f76; }
0126 .st-key-home_flow .home-section { padding-top:0; }
0127 .st-key-home_flow .home-section h2 { color:#fff; }
0128 .st-key-home_flow .home-section p { color:#c9d7f6; }
0129 .home-flow { display:grid; grid-template-columns:repeat(3,1fr); gap:.8rem; }
0130 .home-flow-item { min-height:160px; padding:1.35rem; border-top:1px solid rgba(255,255,255,.28); }
0131 .home-flow-item b { display:block; color:#fff; font-size:1.55rem; margin-bottom:2.2rem; }
0132 .home-flow-item span { color:#c9d7f6; font-size:.92rem; }
0133 [class*='st-key-entry_'] { min-height:180px; margin-bottom:4.5rem; padding:1.15rem; border:1px solid var(--line); border-radius:16px; background:#fff;
transition:transform .3s cubic-bezier(.16,1,.3,1), box-shadow .3s ease, border-color .3s ease; }
0134 [class*='st-key-entry_']:hover { transform:translateY(-5px); border-color:rgba(23,80,223,.3); box-shadow:0 16px 34px rgba(31,57,100,.1); }
0135 [class*='st-key-entry_'] [data-testid='stPageLink-NavLink'] { padding:.65rem 0; border:0!important; box-shadow:none; background:transparent!important;
font-size:1.08rem; }
0136 [class*='st-key-entry_'] [data-testid='stPageLink-NavLink']:hover { transform:none; box-shadow:none; }
0137 .entry-icon { width:42px; height:42px; display:grid; place-items:center; border-radius:12px; background:var(--surface-blue); color:var(--brand); font-weight:900; }
0138 .home-final { margin-top:1.5rem; padding:clamp(1.6rem,4vw,3.8rem); border-radius:24px 24px 0 0; background:linear-gradient(140deg,#102f76,#1854c9); }
0139 .home-final h2 { max-width:18ch; margin:0 0 .8rem; color:#fff; font-size:clamp(2rem,4vw,4rem)!important; line-height:1.04!important; letter-spacing:-.055em; }
0140 .home-final p { margin:0; color:#d7e3ff; }
0141 .st-key-final_upload button { border-radius:0 0 16px 16px; min-height:3.4rem; }
0142 .service-hero { position:relative; overflow:hidden; display:grid; grid-template-columns:1.25fr .75fr; align-items:center; gap:2rem; min-height:310px; margin:1rem 0
2rem; padding:2.4rem; border-radius:20px; background:linear-gradient(145deg,#102f76,#1750df); color:#fff; box-shadow:0 22px 54px rgba(16,47,118,.2); }
0143 .service-hero::after { content:''; position:absolute; inset:auto -80px -160px auto; width:360px; height:360px; border-radius:50%; border:1px solid
rgba(255,255,255,.18); box-shadow:0 0 48px rgba(255,255,255,.04),0 0 96px rgba(255,255,255,.025); animation:network-breathe 5s ease-in-out infinite; }
0144 .service-hero h2 { max-width:12ch; margin:0 0 .8rem; color:#fff; font-size:clamp(2.25rem,4vw,4.3rem)!important; line-height:1.02!important; }
0145 .service-hero p { max-width:38rem; margin:0; color:#dce7ff; font-size:1.05rem; }
0146 .service-orbit { position:relative; z-index:1; min-height:150px; display:grid; place-items:center; text-align:center; border:1px solid rgba(255,255,255,.22);
border-radius:16px; background:rgba(255,255,255,.08); backdrop-filter:blur(14px); }

```

```

0147 .service-orbit span { position:absolute; width:14px; height:14px; border-radius:50%; background:var(--accent); box-shadow:0 0 0 9px rgba(255,129,51,.16);
animation:orbit-order 4.5s linear infinite; }
0148 .service-orbit b { max-width:12ch; color:#fff; font-size:1.08rem; }
0149 .service-assurance { display:grid; grid-template-columns:repeat(3,1fr); margin:3rem 0; border-top:1px solid var(--line); border-bottom:1px solid var(--line); }
0150 .service-assurance div { padding:1.3rem; border-right:1px solid var(--line); }
0151 .service-assurance div:last-child { border-right:0; }
0152 .service-assurance b,service-assurance span { display:block; }
0153 .service-assurance b { color:var(--brand-dark); }
0154 .service-assurance span { margin-top:.35rem; color:var(--muted); font-size:.84rem; }
0155 .fee-preview { display:grid; grid-template-columns:1fr auto; gap:.25rem 1rem; margin:1rem 0; padding:1.25rem 1.35rem; border:1px solid #f0d2be;
border-radius:14px; background:#fff8f3; }
0156 .fee-preview span { color:#7a5a45; font-size:.86rem; }
0157 .fee-preview strong { grid-row:span 2; align-self:center; color:#b45309; font-size:1.75rem; font-variant-numeric:tabular-nums; }
0158 .fee-preview small { max-width:62ch; color:#7a6a60; line-height:1.55; }
0159 .map-order-head { display:grid; grid-template-columns:repeat(3,1fr); margin:1rem 0 1.25rem; border-top:1px solid var(--line); border-bottom:1px solid var(--line); }
0160 .map-order-head>div { padding:1rem 1.2rem; border-right:1px solid var(--line); }
0161 .map-order-head>div:last-child { border-right:0; }
0162 .map-order-head span,map-order-head b { display:block; }
0163 .map-order-head span { color:var(--muted); font-size:.78rem; }
0164 .map-order-head b { margin-top:.35rem; color:var(--brand-dark); font-size:1rem; }
0165 .tracking-head { display:flex; align-items:flex-end; justify-content:space-between; gap:2rem; margin:1rem 0 2rem; padding:2rem; border-radius:18px;
background:#102f76; }
0166 .tracking-head span,tracking-head small { color:#b9caef; font-size:.78rem; }
0167 .tracking-head h2 { margin:.3rem 0; color:#fff; font-size:clamp(2rem,3vw,3.2rem)!important; }
0168 .tracking-head p { margin:0; color:#d8e4ff; }
0169 .tracking-state { min-width:180px; display:grid; gap:.35rem; justify-items:end; }
0170 .tracking-state i { width:12px; height:12px; border-radius:50%; background:var(--accent); box-shadow:0 0 0 rgba(255,129,51,.45); animation:status-pulse 2s
ease-out infinite; }
0171 .tracking-state b { color:#fff; font-size:1.08rem; }
0172 .delivery-track { --track:0%; position:relative; display:grid; grid-template-columns:repeat(5,1fr); margin:1rem 0 3.5rem; padding-top:1rem; }
0173 .delivery-track::before,delivery-track-fill { content:""; position:absolute; top:1.82rem; left:10%; right:10%; height:2px; background:#d7e0ed; }
0174 .delivery-track-fill { right:calc(var(--track) * .8); background:linear-gradient(90deg,var(--brand),var(--accent)); transform-origin:left; animation:track-flow
9s cubic-bezier(.16,1,.3,1) both; }
0175 .delivery-node { position:relative; z-index:1; display:grid; justify-items:center; gap:.6rem; text-align:center; color:var(--muted); }
0176 .delivery-node span { width:28px; height:28px; display:grid; place-items:center; border-radius:50%; background:#e5ebf3; color:#6f7d90; font-size:.72rem;
font-weight:800; transition:transform .3s ease,background .3s ease,color .3s ease; }
0177 .delivery-node.done span { background:var(--brand); color:#fff; transform:scale(1.08); }
0178 .delivery-node.done:last-child span { background:var(--accent); }
0179 .delivery-node b { font-size:.8rem; }
0180 .detail-sheet { background:#fff; border-top:1px solid var(--line); }
0181 .detail-sheet div { display:grid; grid-template-columns:120px 1fr; gap:1rem; padding:1rem .2rem; border-bottom:1px solid var(--line); }
0182 .detail-sheet span { color:var(--muted); font-size:.82rem; }
0183 .detail-sheet b { color:var(--ink); font-size:.92rem; }
0184 .demo-banner { display:flex; align-items:center; justify-content:space-between; gap:1rem; margin-top:2.5rem; padding:1.1rem 1.3rem; border:1px solid #f2cfb7;
border-radius:14px; background:#fff8f3; }
0185 .demo-banner b { color:#b45309; }
0186 .demo-banner span { color:#7a5a45; font-size:.86rem; }
0187 .empty-stage { min-height:300px; display:grid; place-content:center; text-align:center; padding:2rem; border:1px dashed #bdc9db; border-radius:16px;
background:linear-gradient(145deg,#fff,#f0f5ff); }
0188 .empty-stage h2,empty-stage h3 { color:var(--brand-dark); }
0189 .ops-ribbon { position:relative; overflow:hidden; display:flex; align-items:center; gap:1rem; margin:1rem 0 1.5rem; padding:1.3rem 1.5rem; border-radius:16px;
color:#fff; background:#102f76; }
0190 .ops-ribbon span { color:#aebf5; font-size:.82rem; }
0191 .ops-ribbon b { font-size:1.1rem; }
0192 .pulse-route { position:absolute; right:0; width:35%; height:100%; background:linear-gradient(90deg,transparent,rgba(255,255,255,.08)); }
0193 .pulse-route::before { content:""; position:absolute; left:0; right:0; top:50%; height:1px; background:rgba(255,255,255,.25); }
0194 .pulse-route i { position:absolute; top:calc(50% - 5px); width:10px; height:10px; border-radius:50%; background:var(--accent); box-shadow:0 0 18px
rgba(255,129,51,.75); animation:route-run 3s ease-in-out infinite; }
0195 .ops-order { min-height:220px; padding:1.5rem; border-radius:16px; background:linear-gradient(145deg,#edf3ff,#fff); }
0196 .ops-order span { color:var(--accent); font-size:.78rem; font-weight:800; }
0197 .ops-order h3 { margin:.8rem 0 .4rem!important; color:var(--brand-dark); font-size:1.55rem!important; }
0198 .ops-order b { display:block; margin-top:2.3rem; color:var(--brand); }
0199 .integration-rail { display:grid; grid-template-columns:repeat(4,1fr); margin-bottom:2rem; border-top:1px solid var(--line); }
0200 .integration-rail>div { min-height:170px; padding:1.25rem; border-right:1px solid var(--line); }
0201 .integration-rail>div:last-child { border-right:0; }
0202 .integration-rail span,integration-rail b,integration-rail small { display:block; }
0203 .integration-rail span { color:var(--muted); font-size:.78rem; }
0204 .integration-rail b { margin:1.8rem 0 .4rem; color:#b45309; }
0205 .integration-rail .ready b { color:var(--success); }
0206 .integration-rail small { color:var(--muted); line-height:1.5; }
0207 .receipt-head { margin:1rem 0 2rem; padding:2rem; border-radius:18px; background:linear-gradient(145deg,#fff,#edf3ff); }
0208 .receipt-head span { color:var(--brand); font-size:.82rem; font-weight:800; }
0209 .receipt-head h2 { margin:.55rem 0!important; color:var(--brand-dark); font-size:2.4rem!important; }
0210 .motion-focus { animation:focus-arrive .78s cubic-bezier(.16,1,.3,1) both; }
0211 .motion-reveal { animation:reveal-up .55s cubic-bezier(.16,1,.3,1) both; }
0212 @keyframes reveal-up { from { opacity:.01; transform:translateY(16px); filter:blur(2px); } to { opacity:1; transform:none; filter:none; } }
0213 @keyframes reveal-on-scroll { from { opacity:.01; transform:translateY(20px); filter:blur(2px); } to { opacity:1; transform:none; filter:none; } }
0214 @keyframes home-copy { from { opacity:0; transform:translateY(28px); filter:blur(8px); } to { opacity:1; transform:none; filter:none; } }
0215 @keyframes home-visual { from { opacity:0; transform:translateX(34px) scale(.96); clip-path:inset(0 0 100% round 20px); } to { opacity:1; transform:none;
clip-path:inset(0 0 0 round 20px); } }
0216 @keyframes home-ambient { to { transform:translate(-38px,42px) scale(1.08); } }
0217 @keyframes line-grow { from { transform:scaleX(0); opacity:0; } to { transform:scaleX(1); opacity:1; } }
0218 @keyframes focus-arrive { from { opacity:.1; transform:translateY(22px) scale(.985); filter:blur(7px); clip-path:inset(0 0 16% 0 round 16px); } to { opacity:1;
transform:none; filter:none; clip-path:inset(0 0 0 round 16px); } }
0219 @keyframes network-breathe { 50% { transform:scale(1.08); opacity:.76; } }
0220 @keyframes orbit-order { from { transform:rotate(0deg) translateX(62px) rotate(0deg); } to { transform:rotate(360deg) translateX(62px) rotate(-360deg); } }
0221 @keyframes status-pulse { 70% { box-shadow:0 0 12px rgba(255,129,51,0); } 100% { box-shadow:0 0 0 rgba(255,129,51,0); } }
0222 @keyframes track-flow { from { transform:scaleX(0); filter:blur(3px); } to { transform:scaleX(1); filter:none; } }
0223 @keyframes route-run { 0% { left:4%; opacity:0; } 20% { opacity:1; } 80% { opacity:1; } 100% { left:92%; opacity:0; } }

```

```

0224     .metric-card, .route-board, .note-panel, [data-testid='stMetric'], [data-testid='stPlotlyChart'], [data-testid='stDataFrame'] { animation:reveal-up .62s
cubic-bezier(.16,1,,.3,1) both; }
0225     @supports (animation-timeline: view()) { .metric-card, .route-board, .note-panel, .home-reveal, [data-testid='stMetric'], [data-testid='stPlotlyChart'],
[data-testid='stDataFrame'] { animation-name:reveal-on-scroll; animation-duration:1ms; animation-fill-mode:both; animation-timeline:view(); animation-range:entry 5% cover
28%; animation-delay:0ms; } }
0226     @media (max-width: 800px) { h1 { font-size:2rem!important; } .hero-panel { grid-template-columns:1fr; padding:1.3rem; } .hero-index { border-left:0; border-top:1px
solid rgba(255,255,255,.25); padding:1rem 0 0; } .block-container { padding-left:1rem; padding-right:1rem; } .home-nav-copy { display:none; } .st-key-home_hero {
padding:1.2rem; border-radius:20px; } .home-hero-copy h1 { max-width:12ch; font-size:2.75rem!important; } .home-proof { grid-template-columns:repeat(2,1fr);
margin-bottom:4rem; padding:8rem 0; } .home-proof>div { padding:8rem; } .home-proof>div:nth-child(2) { border-right:0; } .home-proof>div:nth-child(-n+2) {
border-bottom:1px solid var(--line); } .home-bento { grid-template-columns:1fr; margin-bottom:4rem; } .home-feature-large { grid-row:auto; min-height:330px; } .home-flow {
grid-template-columns:1fr; } .home-flow-item { min-height:auto; } .home-flow-item b { margin-bottom:5rem; } .st-key-home_flow { padding:1.25rem; } [class*='st-key-entry_'] {
min-height:0; margin-bottom:5rem; } .home-final { border-radius:18px 18px 0 0; } .service-hero { grid-template-columns:1fr; padding:1.4rem; } .service-orbit { min-height:120px;
} .service-assurance { grid-template-columns:1fr; } .service-assurance div { border-right:0; border-bottom:1px solid var(--line); } .fee-preview { grid-template-columns:1fr; } .
fee-preview strong { grid-row:auto; } .map-order-head { grid-template-columns:1fr; } .map-order-head>div { border-right:0; border-bottom:1px solid var(--line); } .
map-order-head>div:last-child { border-bottom:0; } .tracking-head { align-items:flex-start; flex-direction:column; } .tracking-state { justify-items:start; } .delivery-track {
overflow-x:auto; grid-template-columns:repeat(5,110px); padding-bottom:8rem; } .delivery-track::before,.delivery-track-fill { left:55px; right:55px; } .delivery-track-fill {
right:auto; width:calc(var(--track) * 4.4); } .detail-sheet div { grid-template-columns:1fr; gap:.25rem; } .demo-banner { align-items:flex-start; flex-direction:column; } .
integration-rail { grid-template-columns:repeat(2,1fr); } .integration-rail>div:nth-child(2) { border-right:0; } .pulse-route { display:none; } }
0227     @media (prefers-reduced-motion: reduce) { *, *::before, *::after { animation:none!important; transition:none!important; scroll-behavior:auto!important;
transform:none!important; filter:none!important; } }
0228     </style>
0229     """,
0230     unsafe_allow_html=True,
0231 )
0232
0233
0234 def render_sidebar() -> None:
0235     with st.sidebar:
0236         st.markdown("<div class='brand-mark'>YL</div>", unsafe_allow_html=True)
0237         st.markdown("<div class='brand-name'>银犁智慧配送</div>", unsafe_allow_html=True)
0238         st.caption("下单、追踪、地图与签收")
0239         st.divider()
0240         st.markdown("<div class='nav-section-label'>客户服务</div>", unsafe_allow_html=True)
0241         st.page_link("app.py", label="□ 网站首页")
0242         st.page_link("pages/客户下单.py", label="□ 客户下单")
0243         st.page_link("pages/订单追踪.py", label="◉ 订单追踪")
0244         st.page_link("pages/配送网络地图.py", label="◊ 我的配送地图")
0245         st.page_link("pages/电子签收.py", label="□ 确认签收")
0246         st.divider()
0247         st.markdown("<div class='nav-section-label'>工作人员</div>", unsafe_allow_html=True)
0248         if not st.session_state.get("staff_authenticated", False):
0249             password = st.text_input("工作人员密码", type="password", key="staff_password")
0250             if password:
0251                 configured_password = staff_password()
0252                 if not configured_password:
0253                     st.error("未配置工作人员密码。请通过 YL_STAFF_PASSWORD 环境变量配置。")
0254                 elif password == configured_password:
0255                     st.session_state["staff_authenticated"] = True
0256                     st.rerun()
0257                 else:
0258                     st.error("密码不正确")
0259             return
0260         st.caption("已获得工作人员权限")
0261         st.page_link("pages/数据导入与方案生成.py", label="□ 上传订单")
0262         st.page_link("pages/方案驾驶舱.py", label="□ 配送总览")
0263         st.page_link("pages/订单与需求.py", label="□ 订单情况")
0264         st.page_link("pages/运营配送地图.py", label="◊ 配送网络地图")
0265         st.page_link("pages/车辆路径优化.py", label="□ 车辆路径优化")
0266         st.page_link("pages/成本与绩效.py", label="□ 成本与绩效")
0267         st.page_link("pages/方案对比.py", label="□ 方案对比")
0268         st.page_link("pages/企业工作台.py", label="□ 企业工作台")
0269
0270
0271 def require_staff_access() -> None:
0272     if not st.session_state.get("staff_authenticated", False):
0273         st.warning("该页面仅供工作人员使用。请在左侧输入工作人员密码后继续。")
0274         st.stop()
0275
0276
0277 def page_title(title: str, subtitle: str = "") -> None:
0278     st.title(title)
0279     if subtitle:
0280         st.markdown(f"<div class='page-subtitle'>{html.escape(subtitle)}</div>", unsafe_allow_html=True)
0281
0282
0283 def section_label(text: str) -> None:
0284     st.markdown(f"<div class='section-label'>{html.escape(text)}</div>", unsafe_allow_html=True)
0285
0286
0287 def source_note(manifest: dict[str, Any]) -> None:
0288     sources = manifest.get("sources", []) if manifest else []
0289     updated = manifest.get("generated_at", "") if manifest else ""
0290     suffix = f"; 整理时间: {updated}" if updated else ""
0291     st.markdown(f"<div class='source-note'>数据说明: 页面仅展示当前结果文件和已整理的路线数据(suffix). </div>", unsafe_allow_html=True)
0292
0293
0294 def fmt_money(value: Any) -> str:
0295     return "暂无" if value is None or value == "" else f"¥ {float(value):.2f}"
0296
0297

```

```
0298 def fmt_num(value: Any, digits: int = 1) -> str:
0299     return "暂无" if value is None or value == "" else f"{float(value):.{digits}f}"
0300
0301
0302 def plotly_config() -> dict[str, Any]:
0303     return {"displayModeBar": False, "responsive": True}
```

代码附录：pages/客户下单.py

用途：客户下单。当前版本共 85 行。

```
0001 import streamlit as st
0002
0003 from datetime import time
0004
0005 from 功能组件_页面共用代码.order_state import create_order, estimate_delivery_fee, geocode_address, init_orders
0006 from 功能组件_页面共用代码.ui import inject_css, page_title, render_sidebar
0007
0008
0009 inject_css()
0010 render_sidebar()
0011 init_orders()
0012 page_title("客户下单", "填写收货信息和送达时间, 提交前即可查看预估费用")
0013
0014 st.markdown("""
0015 <div class="service-hero motion-focus">
0016   <div><h2>把配送需求交给我们。</h2><p>信息填写完成后, 系统会保存订单并进入备货与车辆安排流程。</p></div>
0017   <div class="service-orbit"><span></span><b>订单正在进入配送网络</b></div>
0018 </div>
0019 """, unsafe_allow_html=True)
0020
0021 st.markdown("### 收货信息")
0022 c1, c2 = st.columns(2)
0023 customer = c1.text_input("客户名称", placeholder="例如: 青羊区某门店")
0024 contact = c2.text_input("联系人", placeholder="收货联系人")
0025 c3, c4 = st.columns([1.4, 1])
0026 address = c3.text_input("收货地址", placeholder="请填写详细地址, 例如: 成都市青羊区人民中路一段")
0027 phone = c4.text_input("联系电话", placeholder="用于配送联系")
0028
0029 if "order_coordinates" not in st.session_state:
0030     st.session_state["order_coordinates"] = None
0031 if st.button("识别地址坐标", disabled=not address.strip()):
0032     coordinates = geocode_address(address)
0033     st.session_state["order_coordinates"] = coordinates
0034     if not coordinates:
0035         st.warning("未识别到坐标。可补充街道和门牌号后重试, 或手工填写下方经纬度。")
0036 coordinates = st.session_state.get("order_coordinates")
0037 geo_left, geo_right = st.columns(2)
0038 manual_lon = geo_left.number_input("经度 (识别失败时可手工填写)", value=float(coordinates[0]) if coordinates else 0.0, format="%.6f")
0039 manual_lat = geo_right.number_input("纬度 (识别失败时可手工填写)", value=float(coordinates[1]) if coordinates else 0.0, format="%.6f")
0040 if manual_lon and manual_lat:
0041     coordinates = (manual_lon, manual_lat, coordinates[2] if coordinates else "手工填写坐标")
0042 if coordinates:
0043     st.success(f"配送坐标: 经度 {coordinates[0]:.6f}, 纬度 {coordinates[1]:.6f}")
0044 else:
0045     st.caption("地址识别为主动请求, 结果缓存 24 小时。未联网时可填写经纬度后继续下单。")
0046
0047 st.markdown("### 配送内容")
0048 p1, p2, p3 = st.columns(3)
0049 product = p1.selectbox("配送品类", ["鲜面条", "姜蒜"])
0050 quantity = p2.number_input("配送重量 (kg)", min_value=1.0, value=100.0, step=10.0)
0051 delivery_date = p3.date_input("期望送达日期")
0052 t1, t2 = st.columns(2)
0053 expected_time = t1.time_input("最早到达时间", value=time(8, 0))
0054 latest_time = t2.time_input("最晚送达时间", value=time(18, 0))
0055 st.caption("费用会随品类、重量、已识别配送坐标和工作人员调价参数实时更新。")
0056 quote = estimate_delivery_fee(product, quantity, coordinates[0] if coordinates else None, coordinates[1] if coordinates else None)
0057 st.markdown(f"""
0058 <div class="fee-preview"><span>本单预估配送费用</span><strong>¥ {quote['预估费用_元']:.2f}</strong><small>起步价 ¥ {quote['起步价_元']:.2f} + 重量价 ¥ {quote['重量
0059 价_元']:.2f}(f + 参考里程价 ¥ {quote['里程价_元']:.2f} ({quote['参考距离_km']:.1f} km) " if coordinates else "。填写坐标后将计入参考里程价。"</small></div>
0060 """, unsafe_allow_html=True)
0061 submitted = st.button("提交配送订单", type="primary", use_container_width=True)
0062
0063 if submitted:
0064     missing = [name for name, value in [("客户名称", customer), ("收货地址", address), ("联系人", contact), ("联系电话", phone)] if not value.strip()]
0065     if missing:
0066         st.error("请补充: " + "、".join(missing))
0067     elif latest_time <= expected_time:
0068         st.error("最晚送达时间需要晚于期望送达时间。")
0069     elif not coordinates:
0070         st.error("请先识别或填写收货地址坐标, 以生成配送路线和完整预估费用。")
0071     else:
0072         order = create_order({
0073             "客户名称": customer.strip(), "联系人": contact.strip(), "联系电话": phone.strip(), "收货地址": address.strip(),
0074             "品类": product, "配送重量_kg": quantity, "期望送达日期": str(delivery_date),
0075             "最早到达": expected_time.strftime("%H:%M"), "最晚到达": latest_time.strftime("%H:%M"),
0076             "期望送达": f"{delivery_date} {expected_time.strftime('%H:%M')}",
0077             "最晚送达": f"{delivery_date} {latest_time.strftime('%H:%M')}", "预估费用_元": quote["预估费用_元"],
0078             "经度": coordinates[0] if coordinates else "", "纬度": coordinates[1] if coordinates else "", "地址识别结果": coordinates[2] if coordinates else "未识别",
0079         })
0080         st.success(f"订单 {order['订单编号']} 已提交, 预估费用 ¥ {order['预估费用_元']:.2f}。演示调度线已生成, 可立即查看追踪和地图。")
0081         st.session_state["active_order_id"] = order["订单编号"]
0082         if order.get("保存状态") == "已保存到本机演示订单表":
0083             st.caption("本机演示订单已保存。客户追踪页只显示当前会话提交的订单。")
0084         else:
```

```
0084         st.warning("订单已进入当前会话，但写入 Excel 失败。工作人员可从当前订单列表导出。")
0085         st.page_link("pages/订单追踪.py", label="查看订单进度", use_container_width=True)
```

代码附录：pages/订单追踪.py

用途：订单追踪。当前版本共 55 行。

```
0001 import streamlit as st
0002
0003 from 功能组件_页面共用代码.order_state import STATUS_FLOW, find_order, init_orders
0004 from 功能组件_页面共用代码.ui import inject_css, page_title, render_sidebar
0005
0006
0007 inject_css()
0008 render_sidebar()
0009 orders = init_orders()
0010 page_title("订单追踪", "查看订单从提交、备货到配送签收的当前进度")
0011
0012 if not orders:
0013     st.markdown("<div class='empty-stage motion-focus'><h2>还没有可追踪的订单。</h2><p>先提交配送需求，订单进度会显示在这里。</p></div>",
0014 unsafe_allow_html=True)
0015     if st.button("创建配送订单", type="primary", use_container_width=True):
0016         st.switch_page("pages/客户下单.py")
0017         st.stop()
0018
0019 ids = [item["订单编号"] for item in orders]
0020 default_id = st.session_state.get("active_order_id", ids[0])
0021 selected = st.selectbox("选择订单", ids, index=ids.index(default_id) if default_id in ids else 0)
0022 order = find_order(selected)
0023 st.session_state["active_order_id"] = selected
0024 index = int(order.get("状态序号", 0))
0025
0026 st.markdown(f"""
0027 <div class="tracking-head motion-focus">
0028 <div><span>订单编号</span><h2>{order['订单编号']}</h2><p>{order['客户名称']} · {order['品类']} · {order['配送重量_kg']}:.0f> kg</p></div>
0029 <div class="tracking-state"><i></i><b>{order['状态']}</b><small>{order['数据模式']}</small></div>
0030 </div>
0031 <div class="delivery-track" style="--track:{index / (len(STATUS_FLOW)-1) * 100.0f}%">
0032 <div class="delivery-track-fill"></div>
0033 {'.join(f'<div class="delivery-node {"done" if i <= index else ""}'><span>{i+1}</span><b>{label}</b></div>' for i, label in enumerate(STATUS_FLOW))}
0034 </div>
0035 """, unsafe_allow_html=True)
0036
0037 left, right = st.columns([1.2, .8], gap="large")
0038 with left:
0039     st.markdown("### 配送信息")
0040     st.markdown(f"""
0041 <div class="detail-sheet motion-reveal">
0042 <div><span>收货地址</span><b>{order['收货地址']}</b></div>
0043 <div><span>期望送达</span><b>{order['期望送达']}</b></div>
0044 <div><span>最晚送达</span><b>{order['最晚送达']}</b></div>
0045 <div><span>预估费用</span><b>¥ {float(order.get('预估费用_元', 0)):.2f}</b></div>
0046 <div><span>配送安排</span><b>{'演示调度线已生成' if order.get('路线GeoJSON') else '待补充收货坐标'}</b></div>
0047 </div>
0048 """, unsafe_allow_html=True)
0049 with right:
0050     st.markdown("### 配送查看")
0051     st.info("演示模式下可查看订单状态与调度线；正式运行时将以调度系统的车辆和到达信息为准。")
0052     st.page_link("pages/配送网络地图.py", label="查看我的配送地图", use_container_width=True)
0053     if index == len(STATUS_FLOW) - 1 and st.button("完成电子签收", type="primary", use_container_width=True):
0054         st.switch_page("pages/电子签收.py")
0055
0056 st.markdown("<div class='demo-banner motion-reveal'><b>本机演示模式</b><span>订单状态和调度线用于流程演示，不展示虚构的实时车辆位置或温度数据。</span></div>", unsafe_allow_html=True)
```

代码附录：pages/配送网络地图.py

用途：客户地图。当前版本共 50 行。

```
0001 import folium
0002 import streamlit as st
0003 from streamlit_folium import st_folium
0004
0005 from 功能组件_页面共用代码.maps import add_zoom_detail_behavior, create_chengdu_map
0006 from 功能组件_页面共用代码.order_state import find_order, init_orders
0007 from 功能组件_页面共用代码.ui import inject_css, page_title, render_sidebar
0008
0009
0010 inject_css()
0011 render_sidebar()
0012 orders = init_orders()
0013 page_title("我的配送地图", "仅显示当前订单的配送路线；放大地图后自动显示细支道路")
0014
0015 if not orders:
0016     st.markdown("<div class='empty-stage motion-focus'><h2>还没有可查看路线的订单。</h2><p>提交配送订单后，调度确认的路线会显示在这里。</p></div>",
0017 unsafe_allow_html=True)
0018     if st.button("前往客户下单", type="primary", use_container_width=True):
0019         st.switch_page("pages/客户下单.py")
0020         st.stop()
0021
0022 ids = [item["订单编号"] for item in orders]
0023 active = st.session_state.get("active_order_id", ids[0])
0024 selected = st.selectbox("查看订单", ids, index=ids.index(active) if active in ids else 0)
0025 order = find_order(selected)
0026 st.session_state["active_order_id"] = selected
0027
0028 st.markdown(f"""
0029 <div class="map-order-head motion-focus">
0030 <div><span>当前订单</span><b>{order['订单编号']}</b></div>
0031 <div><span>配送品类</span><b>{order['品类']}</b></div>
0032 <div><span>当前状态</span><b>{order['状态']}</b></div>
0033 </div>
0034 """, unsafe_allow_html=True)
0035
0036 fmap, minor_layer = create_chengdu_map(zoom_start=10)
0037 route = order.get("路线GeoJSON")
0038 if route and route.get("features"):
0039     folium.GeoJson(
0040         route,
0041         name="本订单配送路线",
0042         style_function=lambda _: {"color": "#1750df", "weight": 5, "opacity": 0.9},
0043         tooltip="本订单配送路线",
0044     ).add_to(fmap)
0045 else:
0046     st.info("该订单尚未取得坐标，暂不能生成演示调度线。请返回订单页补充经纬度。")
0047
0048 add_zoom_detail_behavior(fmap, minor_layer, threshold=13)
0049 folium.LayerControl(collapsed=True, position="topright").add_to(fmap)
0050 st_folium(fmap, use_container_width=True, height=650, returned_objects=[])
0051 st.caption("地图使用团队整理的道路数据。蓝线为本订单的演示调度线，用于串联订单流程，不等于正式 Dijkstra 路网最短路径。")
```

代码附录：pages/电子签收.py

用途：电子签收。当前版本共 63 行。

```
0001 import streamlit as st
0002
0003 from 功能组件_页面共用代码.order_state import STATUS_FLOW, find_order, init_orders, persist_order
0004 from 功能组件_页面共用代码.ui import inject_css, page_title, render_sidebar
0005
0006
0007 inject_css()
0008 render_sidebar()
0009 orders = init_orders()
0010 page_title("确认签收", "正常到货一键确认；发现异常可在下方提交反馈")
0011
0012 if not orders:
0013     st.info("当前没有可签收的订单。")
0014     if st.button("返回客户下单", type="primary", use_container_width=True):
0015         st.switch_page("pages/客户下单.py")
0016     st.stop()
0017
0018 ids = [item["订单编号"] for item in orders]
0019 active = st.session_state.get("active_order_id", ids[0])
0020 selected = st.selectbox("选择订单", ids, index=ids.index(active) if active in ids else 0)
0021 order = find_order(selected)
0022
0023 st.markdown(f"<div class='receipt-head motion-focus'><span>{order['订单编号']}</span><h2>{order['客户名称']}</h2><p>{order['品类']} · {order['配送重量_kg']:.0f} kg · {order['收货地址']}</p></div>", unsafe_allow_html=True)
0024
0025 if order.get("状态") == "配送完成":
0026     st.success("该订单已确认签收。")
0027 elif order.get("状态") != "配送途中":
0028     st.info(f"当前订单处于 \"{order.get('状态')}\", 待配送车辆发出后可确认签收。")
0029 else:
0030     st.markdown("### 货物确认无误")
0031     st.caption("请在确认数量、包装和货物状态无异常后完成签收。")
0032     if st.button("确认签收", type="primary", use_container_width=True):
0033         order["状态"] = "配送完成"
0034         order["状态序号"] = 4
0035         order["签收结果"] = "正常签收"
0036         persist_order(order)
0037         st.success("签收完成，订单状态已更新。")
0038         st.rerun()
0039
0040 st.divider()
0041 with st.expander("异常反馈", expanded=False):
0042     st.caption("包装、数量、温度或送达时间存在问题时，请提交情况说明。")
0043     with st.form("exception_form"):
0044         problems = st.multiselect("异常类型", ["包装破损", "数量不符", "温度异常", "送达延迟", "其他"])
0045         detail = st.text_area("异常情况说明", placeholder="请描述发现的问题，便于工作人员尽快处理")
0046         callback = st.checkbox(f"需要客服回电至 {order.get('联系电话', '订单联系电话')}")
0047         submitted = st.form_submit_button("提交异常反馈", use_container_width=True)
0048     if submitted:
0049         if not problems or not detail.strip():
0050             st.error("请选择异常类型并填写情况说明。")
0051         else:
0052             order["异常反馈"] = {
0053                 "异常类型": "、".join(problems),
0054                 "情况说明": detail.strip(),
0055                 "需要回电": callback,
0056                 "处理状态": "等待客服联系",
0057             }
0058             persist_order(order)
0059             st.success("异常反馈已提交，工作人员将根据订单联系电话与您沟通。")
0060
0061 if order.get("异常反馈"):
0062     feedback = order["异常反馈"]
0063     st.info(f"已提交: {feedback['异常类型']} | {feedback['处理状态']}")
```

代码附录：pages/企业工作台.py

用途：企业工作台。当前版本共 59 行。

```
0001 import pandas as pd
0002 import streamlit as st
0003
0004 from 功能组件_页面共用代码.data_loader import load_site_data
0005 from 功能组件_页面共用代码.order_state import advance_order, init_orders, pricing_settings, save_pricing_settings
0006 from 功能组件_页面共用代码.ui import fmt_money, inject_css, page_title, render_sidebar, require_staff_access
0007
0008
0009 inject_css()
0010 render_sidebar()
0011 require_staff_access()
0012 data = load_site_data()
0013 orders = init_orders(include_saved=True)
0014 page_title("企业工作台", "集中查看演示订单、配送进度与调度路线")
0015
0016 summary = data.summary.get("noodle", {})
0017 st.markdown("""
0018 <div class="ops-ribbon motion-focus"><span>今日运营</span><b>从订单进入，到配送完成</b></div><div class="pulse-route"><i></i></div></div>
0019 """, unsafe_allow_html=True)
0020
0021 metrics = st.columns(4)
0022 metrics[0].metric("本机演示订单", len(orders))
0023 metrics[1].metric("参考可用车辆", summary.get("vehicles_used", "暂无"))
0024 metrics[2].metric("参考配送里程", f'{summary.get("total_distance_km", 0):.1f} km')
0025 metrics[3].metric("参考方案费用", fmt_money(summary.get("total_cost")))
0026
0027 st.markdown("### 订单流转")
0028 if not orders:
0029     st.markdown("<div class='empty-stage motion-reveal'><h3>当前没有客户提交的订单</h3><p>客户下单后，订单会自动进入这里。</p></div>",
0030 unsafe_allow_html=True)
0031 else:
0032     labels = [f'{item["订单编号"]} · {item["客户名称"]} · {item["状态"]}' for item in orders]
0033     chosen_label = st.selectbox("选择需要处理的订单", labels)
0034     chosen = orders[labels.index(chosen_label)]
0035     c1, c2 = st.columns([1, 2])
0036     with c1:
0037         st.markdown(f"<div class='ops-order motion-reveal'><span>{chosen['品类']}</span><h3>{chosen['客户名称']}</h3><p>{chosen['配送重量_kg']:.0f} kg · {chosen['期望送达']}</p><b>{chosen['状态']}</b></div>", unsafe_allow_html=True)
0038         if st.button("推进到下一状态", type="primary", use_container_width=True, disabled=chosen["状态"] == "配送完成"):
0039             advance_order(chosen)
0040             st.rerun()
0041     with c2:
0042         frame = pd.DataFrame(orders)
0043         show = [c for c in ["订单编号", "客户名称", "品类", "配送重量_kg", "期望送达", "状态"] if c in frame.columns]
0044         st.dataframe(frame[show], use_container_width=True, hide_index=True)
0045
0046 st.info("演示模式：订单、状态、报价和调度线在本机保存；正式路线、库存与车辆设备数据仍以竞赛模型及后续业务系统为准。")
0047
0048 st.markdown("### 配送调价")
0049 st.caption("修改后，客户下单页的预估费用会立即采用新参数。该参数仅用于预估报价，正式路线成本仍以 MATLAB 模型或调度核算为准。")
0050 rates = pricing_settings()
0051 with st.form("pricing_form"):
0052     a, b = st.columns(2)
0053     start_price = a.number_input("起步价 (元) ", min_value=0.0, value=float(rates["起步价_元"]), step=1.0)
0054     distance_price = b.number_input("参考里程价 (元/km) ", min_value=0.0, value=float(rates["里程价_元每km"]), step=0.1)
0055     c, d = st.columns(2)
0056     noodle_price = c.number_input("鲜面条重量价 (元/kg) ", min_value=0.0, value=float(rates["鲜面条_元每kg"]), step=0.01, format="%.2f")
0057     ginger_price = d.number_input("姜蒜重量价 (元/kg) ", min_value=0.0, value=float(rates["姜蒜_元每kg"]), step=0.01, format="%.2f")
0058     if st.form_submit_button("保存调价参数", type="primary", use_container_width=True):
0059         save_pricing_settings({"起步价_元": start_price, "里程价_元每km": distance_price, "鲜面条_元每kg": noodle_price, "姜蒜_元每kg": ginger_price})
0060         st.success("调价参数已保存，后续订单将按新参数预估。")
```

代码附录：pages/运营配送地图.py

用途：运营地图。当前版本共 28 行。

```
0001 import folium
0002 import streamlit as st
0003 from streamlit_folium import st_folium
0004
0005 from 功能组件_页面共用代码.maps import add_customer_points, add_route_features, add_zoom_detail_behavior, create_chengdu_map
0006 from 功能组件_页面共用代码.data_loader import load_site_data
0007 from 功能组件_页面共用代码.order_state import init_orders
0008 from 功能组件_页面共用代码.ui import inject_css, page_title, render_sidebar, require_staff_access
0009
0010
0011 inject_css()
0012 render_sidebar()
0013 require_staff_access()
0014 page_title("运营配送地图", "查看竞赛方案的全量配送网络，并叠加本机演示订单调度线")
0015
0016 data = load_site_data()
0017 orders = init_orders(include_saved=True)
0018 fmap, minor_layer = create_chengdu_map(zoom_start=10)
0019 add_route_features(fmap, data.route_features)
0020 add_customer_points(fmap, data.customer_points)
0021 for order in orders:
0022     route = order.get("路线GeoJSON")
0023     if route and route.get("features"):
0024         folium.GeoJson(route, name=f"演示订单 {order['订单编号']}", style_function=lambda _: {"color": "#ff8133", "weight": 4, "opacity": 0.9}, tooltip=f"演示订单: {order['订单编号']}").add_to(fmap)
0025 add_zoom_detail_behavior(fmap, minor_layer, threshold=13)
0026 folium.LayerControl(collapsed=True).add_to(fmap)
0027 st_folium(fmap, use_container_width=True, height=650, returned_objects=[])
0028 st.caption("蓝色线路为题目数据中的配送网络，橙色线路为本机演示订单的坐标调度线。两者均不表示实时车辆位置。")
```

代码附录：pages/方案驾驶舱.py

用途：方案驾驶舱。当前版本共 77 行。

```
0001 import streamlit as st
0002 import plotly.express as px
0003 import folium
0004 from streamlit_folium import st_folium
0005
0006 from 功能组件_页面共用代码.data_loader import load_site_data, get_summary
0007 from 功能组件_页面共用代码.maps import add_customer_points, add_route_features, add_zoom_detail_behavior, create_chengdu_map
0008 from 功能组件_页面共用代码.ui import fmt_money, fmt_num, inject_css, page_title, plotly_config, render_sidebar, require_staff_access, section_label, source_note
0009
0010 inject_css()
0011 render_sidebar()
0012 require_staff_access()
0013 data = load_site_data()
0014 page_title("配送总览", "先看结果，再查看路线、车辆和费用")
0015
0016 scenario = st.selectbox("查看方案", ["鲜面条配送", "姜蒜配送", "车辆复用"], index=0)
0017 key = {"鲜面条配送": "noodle", "姜蒜配送": "ginger", "车辆复用": "reuse"}[scenario]
0018 summary = get_summary(data, key)
0019
0020 if not summary:
0021     st.warning("暂无可验证数据。请先运行 数据整理脚本_生成网站数据/prepare_data.py。")
0022 else:
0023     cols = st.columns(5)
0024     metrics = [
0025         ("服务客户", summary.get("customers")),
0026         ("配送车辆", summary.get("vehicles_used")),
0027         ("预计里程", f"{summary.get('total_distance_km', 0):.1f} km"),
0028         ("预计费用", fmt_money(summary.get("total_cost"))),
0029         ("预计准时送达", f"{summary.get('ontime_rate', 0) * 100:.0f}%"),
0030     ]
0031     for col, (label, value) in zip(cols, metrics):
0032         col.metric(label, value if value is not None else "暂无")
0033
0034     section_label("结果解读")
0035     left, right = st.columns([1.1, 1], gap="large")
0036     with left:
0037         plain = {"noodle": "面条订单按客户需求和送达时间安排车辆。", "ginger": "姜蒜订单按品类需求和送达时间安排车辆。", "reuse": "两类订单按照车辆可用时间安排配送任务。"}
0038         st.markdown(f"### (plain.get(key, '当前配送方案'))")
0039         load_text = f", 共配送 {(fmt_num(summary.get('total_load_kg'), 0))} kg" if summary.get("total_load_kg") is not None else ""
0040         st.markdown(f"当前方案服务 {(summary.get('customers', '暂无'))} 个客户，安排 {(summary.get('vehicles_used', '暂无'))} 辆车，预计完成 {(summary.get('total_distance_km', 0):.1f)} km" + load_text)
0041         checks = summary.get("validations", {})
0042         if checks:
0043             st.markdown("#### 服务检查")
0044             check_names = {"all_customers_once": "客户覆盖完整", "capacity": "车辆没有超载", "coordinates_available": "客户位置完整", "vehicle_time_no_overlap": "车辆时段不重叠", "vehicle_time_no_conflict": "车辆时间不冲突", "跨产品复用已记录": "复用安排已记录"}
0045             check_cols = st.columns(3)
0046             for col, (name, passed) in zip(check_cols * 3, list(checks.items())[:9]):
0047                 label = check_names.get(name, name)
0048                 status_class = "status-good" if passed else "status-warn"
0049                 status_text = "已确认" if passed else "需复核"
0050                 col.markdown(f"<span class='{status_class}'>{status_text}</span> - {label}</span>", unsafe_allow_html=True)
0051             with st.expander("查看计算说明"):
0052                 st.caption(summary.get("solution_type", "页面读取已保存的配送结果。"))
0053     with right:
0054         cost = summary.get("cost_breakdown", {})
0055         if not cost:
0056             cost = {k: summary.get(k) for k in ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"] if summary.get(k) is not None}
0057         if cost:
0058             labels = {"fixed_cost": "车辆固定费用", "transport_cost": "运输费用", "cooling_cost": "冷藏费用", "time_penalty_cost": "时间影响费用", "loss_cost": "货损费用", "variable_cost": "运行费用"}
0059             frame = {"费用项目": [labels.get(k, k) for k in cost], "费用 (元)": list(cost.values())}
0060             fig = px.bar(frame, x="费用 (元)", y="费用项目", orientation="h", text_auto=".0f", color_discrete_sequence=["#ff8133"])
0061             fig.update_layout(margin=dict(l=0, r=0, t=16, b=0), height=260, paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"), showlegend=False)
0062             st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0063         else:
0064             st.info("成本分项暂无可验证数据。")
0065
0066     source_note(data.manifest)
0067
0068     section_label("全部配送路线")
0069     map_filter = st.radio("地图显示", ["全部路线", "鲜面专线", "姜蒜专线"], horizontal=True)
0070     product_code = {"全部路线": None, "鲜面专线": "N", "姜蒜专线": "G"}[map_filter]
0071     fmap, minor_layer = create_chengdu_map(zoom_start=10)
0072     add_route_features(fmap, data.geojson.get("routes", {}), product_code)
0073     add_customer_points(fmap, data.geojson.get("customers", {}), product_code)
0074     add_zoom_detail_behavior(fmap, minor_layer, threshold=13)
0075     folium.LayerControl(collapsed=True, position="topright").add_to(fmap)
0076     st_folium(fmap, use_container_width=True, height=620, returned_objects=[])
0077     st.caption("工作人员可按品类查看全部配送路径和客户点；放大地图后自动显示细支道路。")
```

代码附录：pages/订单与需求.py

用途：订单与需求。当前版本共 58 行。

```
0001 import streamlit as st
0002 import plotly.express as px
0003
0004 from 功能组件_页面共用代码.data_loader import load_site_data
0005 from 功能组件_页面共用代码.ui import inject_css, page_title, plotly_config, render_sidebar, require_staff_access, section_label, source_note
0006
0007 inject_css()
0008 render_sidebar()
0009 require_staff_access()
0010 data = load_site_data()
0011 page_title("订单概览", "先看订单总量，再按品类和需求筛选")
0012
0013 if data.customers.empty:
0014     st.warning("暂无客户明细。请先运行 数据整理脚本_生成网站数据/prepare_data.py。")
0015 else:
0016     frame = data.customers.copy()
0017     categories = ["全部"] + sorted(frame["产品"].dropna().astype(str).unique().tolist()) if "产品" in frame else ["全部"]
0018     category = st.selectbox("产品筛选", categories)
0019     if category != "全部":
0020         frame = frame[frame["产品"] == category]
0021     if "需求量_kg" in frame:
0022         low, high = int(frame["需求量_kg"].min()), int(frame["需求量_kg"].max())
0023         demand = st.slider("需求量范围 (kg) ", low, max(high, low + 1), (low, high))
0024         frame = frame[frame["需求量_kg"].between(*demand)]
0025
0026     cols = st.columns(4)
0027     for col, (label, value) in zip(cols, [(("当前客户", len(frame)), ("总需求量", f"{frame['需求量_kg'].sum():.0f} kg" if "需求量_kg" in frame else "暂无"), ("平均每客户", f"{frame['需求
量_kg'].mean():.1f} kg" if "需求量_kg" in frame else "暂无"), ("配送品类", frame["产品"].nunique() if "产品" in frame else "暂无"))]):
0028         col.metric(label, value)
0029
0030     section_label("订单结构")
0031     left, right = st.columns(2, gap="large")
0032     with left:
0033         if "产品" in frame and "需求量_kg" in frame:
0034             product = frame.groupby("产品", as_index=False)["需求量_kg"].sum()
0035             fig = px.pie(product, names="产品", values="需求量_kg", hole=.58, color="产品", color_discrete_map={"鲜面条": "#1750df", "姜蒜": "#ff8133"})
0036             fig.update_traces(textposition="outside", texttemplate="%{(label)<br>%(value:,.0f) kg", hovertemplate="%{(label): %(value:,.1f) kg <extra> </extra> ")
0037             fig.update_layout(height=330, margin=dict(l=10, r=10, t=16, b=10), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"),
showlegend=False)
0038             st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0039     with right:
0040         if "时间窗" in frame.columns:
0041             frame["送达时段"] = frame["时间窗"].astype(str).str[:3].map(lambda value: f"{int(value) // 60:02d}:{int(value) % 60:02d}" if value.isdigit() else value)
0042             order = sorted(frame["送达时段"].dropna().unique().tolist())
0043             fig = px.histogram(frame, x="送达时段", color="产品" if "产品" in frame else None, barmode="group", category_orders={"送达时段": order}, color_discrete_map={"鲜
面条": "#1750df", "姜蒜": "#ff8133"})
0044             fig.update_layout(height=330, margin=dict(l=10, r=10, t=16, b=10), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"),
legend_title_text="")
0045             fig.update_xaxes(title="最早送达时间")
0046             st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0047         else:
0048             st.info("时间窗分布暂无可验证数据。")
0049
0050     section_label("客户清单")
0051     st.caption("以下只显示业务确认需要的字段，完整数据仍保留在网站数据文件中。")
0052     view_columns = ["客户编号", "产品", "经度", "纬度", "需求量_kg", "时间窗", "服务时间_分"]
0053     view = frame[[column for column in view_columns if column in frame.columns]].copy()
0054     view = view.rename(columns={"需求量_kg": "需求量 (kg)", "时间窗": "期望送达时间", "服务时间_分": "服务时间 (分钟)"})
0055     with st.expander("查看客户清单", expanded=False):
0056         st.dataframe(view, use_container_width=True, hide_index=True)
0057
0058 source_note(data.manifest)
```

代码附录：pages/车辆路径优化.py

用途：路径优化。当前版本共 56 行。

```
0001 import streamlit as st
0002 import plotly.express as px
0003
0004 from 功能组件_页面共用代码.data_loader import load_site_data
0005 from 功能组件_页面共用代码.ui import fmt_money, fmt_num, inject_css, page_title, plotly_config, render_sidebar, require_staff_access, section_label, source_note
0006
0007 inject_css()
0008 render_sidebar()
0009 require_staff_access()
0010 data = load_site_data()
0011 page_title("车辆安排", "查看每辆车服务哪些客户、走多远、预计花费多少")
0012
0013 scenario = st.radio("配送品类", ["鲜面条配送", "姜蒜配送"], horizontal=True)
0014 key = "noodle" if scenario == "鲜面条配送" else "ginger"
0015 routes = data.routes.get(key)
0016 arrivals = data.arrivals.get(key)
0017 summary = data.summary.get(key, {})
0018
0019 if routes is None or routes.empty:
0020     st.warning("暂无路线结果数据。请先运行 数据整理脚本_生成网站数据/prepare_data.py。")
0021 else:
0022     vehicle_options = ["全部车辆"] + routes["车辆编号"].astype(str).tolist() if "车辆编号" in routes else ["全部车辆"]
0023     vehicle = st.selectbox("查看车辆", vehicle_options)
0024     selected = routes if vehicle == "全部车辆" else routes[routes["车辆编号"].astype(str) == vehicle]
0025     cols = st.columns(4)
0026     for col, (label, value) in zip(cols, [(("配送路线", len(selected)), ("服务客户", int(selected["客户数"].sum())) if "客户数" in selected else "暂无"), ("预计里程", f"{selected['总里程_km'].sum():.1f} km" if "总里程_km" in selected else "暂无"), ("预计费用", fmt_money(selected["总成本_元"].sum())) if "总成本_元" in selected else "暂无"))]:
0027         col.metric(label, value)
0028
0029     section_label("车辆安排")
0030     display_cols = ["车辆编号", "车型", "客户数", "配送量_kg", "装载率", "发车时刻", "回场时刻", "总里程_km", "总成本_元", "访问顺序"]
0031     view = selected[[c for c in display_cols if c in selected.columns]].copy().rename(columns={"客户数": "服务客户数", "配送量_kg": "配送量 (kg)", "发车时刻": "出发时间", "回场时刻": "回场时间", "总里程_km": "里程 (km)", "总成本_元": "费用 (元)", "访问顺序": "配送顺序"})
0032     st.dataframe(view, use_container_width=True, hide_index=True)
0033
0034     left, right = st.columns(2, gap="large")
0035     with left:
0036         if "装载率" in selected:
0037             fig = px.bar(selected, x="车辆编号", y="装载率", text=selected["装载率"].map(lambda v: f"{v:.0%}"), color_discrete_sequence=["#1750df"])
0038             fig.update_yaxes(tickformat=".0%", range=[0, 1.05])
0039             fig.update_layout(height=320, margin=dict(l=0, r=0, t=16, b=0), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"),
0040 yaxis_title="车辆装载率", xaxis_title="车辆")
0041             st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0042     with right:
0043         hist = data.history.get(key, {})
0044         if hist.get("history"):
0045             with st.expander("查看系统计算过程", expanded=False):
0046                 h = hist["history"]
0047                 fig = px.line(h, x="iteration", y="best_cost", markers=True, color_discrete_sequence=["#ff8133"])
0048                 fig.update_layout(height=280, margin=dict(l=0, r=0, t=16, b=0), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"),
0049 xaxis_title="计算轮次", yaxis_title="当前费用")
0050                 st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0051         else:
0052             st.info("算法迭代记录暂无可验证数据。")
0053
0054     if arrivals is not None and not arrivals.empty:
0055         section_label("客户送达情况")
0056         st.dataframe(arrivals, use_container_width=True, hide_index=True)
0057
0058 source_note(data.manifest)
```

代码附录：pages/成本与绩效.py

用途：成本绩效。当前版本共 46 行。

```
0001 import streamlit as st
0002 import plotly.express as px
0003
0004 from 功能组件_页面共用代码.data_loader import load_site_data
0005 from 功能组件_页面共用代码.ui import fmt_money, inject_css, page_title, plotly_config, render_sidebar, require_staff_access, section_label, source_note
0006
0007 inject_css()
0008 render_sidebar()
0009 require_staff_access()
0010 data = load_site_data()
0011 page_title("费用与服务", "看清这次配送的费用构成和服务表现")
0012
0013 scenario = st.selectbox("查看方案", ["鲜面条配送", "姜蒜配送", "车辆复用"])
0014 key = {"鲜面条配送": "noodle", "姜蒜配送": "ginger", "车辆复用": "reuse"}[scenario]
0015 summary = data.summary.get(key, {})
0016
0017 if not summary:
0018     st.warning("暂无可验证数据。")
0019 else:
0020     cost = summary.get("cost_breakdown", {}) or {k: summary.get(k) for k in ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"]} if
summary.get(k) is not None)
0021     section_label("费用构成")
0022     if cost:
0023         cost_labels = {"fixed_cost": "车辆固定费用", "transport_cost": "运输费用", "cooling_cost": "冷藏费用", "time_penalty_cost": "时间影响费用", "loss_cost": "货损费用", "
variable_cost": "运行费用"}
0024         frame = {"费用项目": [cost_labels.get(k, k) for k in cost], "费用 (元)": list(cost.values())}
0025         fig = px.bar(frame, y="费用项目", x="费用 (元)", orientation="h", text_auto=".2f", color_discrete_sequence=["#ff8133"])
0026         fig.update_layout(height=280, margin=dict(l=0,r=0,t=12,b=0), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"), xaxis_title="
金额 (元)", yaxis_title="")
0027         st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0028
0029 cols = st.columns(4)
0030 for col, (label, value) in zip(cols, [{"预计总费用", fmt_money(summary.get("total_cost"))}, {"每公里费用", fmt_money(summary.get("total_cost", 0) /
summary.get("total_distance_km", 1)) if summary.get("total_distance_km") else "暂无"}, {"按时送达客户", summary.get("ontime_customers", "暂无")}, {"平均装载率",
f"{summary.get('average_load_rate', 0) * 100:.1f}%" if summary.get("average_load_rate") is not None else "暂无"}]):
0031     col.metric(label, value)
0032
0033 section_label("车辆成本参考")
0034 if data.sensitivity.empty:
0035     st.info("暂无灵敏度分析数据。")
0036 else:
0037     sens = data.sensitivity.copy()
0038     x = "满载能耗增幅" if "满载能耗增幅" in sens else sens.columns[0]
0039     y = "修正后总成本上界_元" if "修正后总成本上界_元" in sens else sens.columns[-1]
0040     fig = px.line(sens, x=x, y=y, markers=True, color_discrete_sequence=["#1750df"])
0041     fig.update_layout(height=320, margin=dict(l=0,r=0,t=12,b=0), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"))
0042     st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0043     with st.expander("查看详细参考数据", expanded=False):
0044         st.dataframe(sens, use_container_width=True, hide_index=True)
0045
0046 source_note(data.manifest)
```

代码附录：pages/方案对比.py

用途：方案对比。当前版本共 39 行。

```
0001 import pandas as pd
0002 import plotly.express as px
0003 import streamlit as st
0004
0005 from 功能组件_页面共用代码.data_loader import load_site_data
0006 from 功能组件_页面共用代码.ui import inject_css, page_title, plotly_config, render_sidebar, require_staff_access, section_label, source_note
0007
0008 inject_css()
0009 render_sidebar()
0010 require_staff_access()
0011 data = load_site_data()
0012 page_title("方案比较", "把不同配送安排放在一起, 帮助客户和企业做选择")
0013
0014 rows = []
0015 for key, name in [("noodle", "鲜面条配送"), ("ginger", "姜蒜专线"), ("reuse", "车辆复用")]:
0016     s = data.summary.get(key, {})
0017     if s:
0018         rows.append(("方案": name, "客户数": s.get("customers"), "启用车辆": s.get("vehicles_used"), "总里程_km": s.get("total_distance_km"), "总成本_元": s.get("total_cost"), "准
0019 时率": s.get("ontime_rate"), "总载重_kg": s.get("total_load_kg")))
0020
0021 if not rows:
0022     st.warning("暂无可比较的方案数据。")
0023 else:
0024     frame = pd.DataFrame(rows)
0025     section_label("关键指标")
0026     view = frame.rename(columns={"总里程_km": "预计里程 (km)", "总成本_元": "预计费用 (元)", "启用车辆": "配送车辆", "准时率": "预计准时送达"})
0027     st.dataframe(view.style.format({"预计里程 (km)": "{:,1f}", "预计费用 (元)": "¥{:,2f}", "预计准时送达": "{:0%}", "总载重_kg": "{:,0f}"}, na_rep="暂无"),
0028 use_container_width=True, hide_index=True)
0029     metric = st.selectbox("选择要比较的内容", ["总成本_元", "总里程_km", "启用车辆", "准时率"], format_func=("总成本_元": "预计费用", "总里程_km": "预计里程", "启用车辆": "配送
0030 车辆", "准时率": "预计准时送达").get)
0031     labels = {"总成本_元": "预计费用 (元)", "总里程_km": "预计里程 (km)", "启用车辆": "配送车辆", "准时率": "预计准时送达"}
0032     fig = px.bar(frame, x="方案", y=metric, text_auto=".1%" if metric == "准时率" else ".1f", color="方案", color_discrete_sequence=["#1750df", "#ff8133", "#7b8798"])
0033     if metric == "准时率":
0034         fig.update_yaxes(tickformat=".0%")
0035     fig.update_yaxes(title=labels[metric])
0036     fig.update_layout(height=350, margin=dict(l=0,r=0,t=12,b=0), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"),
0037 showlegend=False)
0038     st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0039
0040 section_label("怎么选")
0041 st.markdown("如果更关注单一品类的服务稳定性, 可以查看独立配送方案; 如果更关注车辆利用和整体费用, 可以进一步查看车辆复用方案。实际使用前请结合当天订单和交通情况
0042 确认。")
0043
0044 source_note(data.manifest)
```

代码附录：pages/数据导入与方案生成.py

用途：上传与生成。当前版本共 105 行。

```
0001 import streamlit as st
0002
0003 from 功能组件_页面共用代码.matlab_bridge import run_noodle_model
0004 from 功能组件_页面共用代码.planner import generate_plan, is_result_json, normalize_customers, read_uploaded_file, result_json_to_plan
0005 from 功能组件_页面共用代码.ui import fmt_money, inject_css, page_title, plotly_config, render_sidebar, require_staff_access, section_label, source_note
0006
0007 inject_css()
0008 render_sidebar()
0009 require_staff_access()
0010 page_title("数据导入与方案生成", "上传订单后，快速得到一份清晰的配送安排")
0011
0012 st.markdown("""
0013 <div class="hero-panel">
0014   <div>
0015     <div class="hero-kicker">上传订单 · 自动整理</div>
0016     <h2>上传订单，直接得到配送安排。</h2>
0017     <p>支持 Excel、CSV 和已保存的结果 JSON。系统会整理车辆、配送顺序、预计到达和费用，结果只保存在当前会话。</p>
0018   </div>
0019   <div class="hero-index"><span>结果内容</span><strong>路线 · 车辆 · 费用</strong><small>可下载配送清单</small></div>
0020 </div>
0021 """, unsafe_allow_html=True)
0022
0023 uploaded_files = st.file_uploader("上传客户订单或结果文件", type=["xlsx", "xls", "csv", "json"], accept_multiple_files=True, help="客户表至少包含客户编号、经度、纬度、需求量。")
0024
0025 product = st.selectbox("配送品类", ["鲜面条", "姜蒜"], index=0)
0026
0027 st.caption("如果上传的是客户订单，系统会按以下默认值快速生成一份配送草案。")
0028 with st.expander("正式 MATLAB 模型", expanded=False):
0029     st.write("该入口运行团队的 `solve_first_question_noodle.m`。它仅适用于题目原始的 30 个鲜面条客户、最短距离矩阵和车辆参数，不会把任意上传表伪装成正式求解。")
0030     if st.button("运行正式鲜面条 MATLAB 模型", use_container_width=True):
0031         with st.spinner("MATLAB 正在执行多起点构造与邻域搜索..."):
0032             ok, message = run_noodle_model()
0033             (st.success if ok else st.error)(message)
0034 with st.expander("调整计算参数", expanded=False):
0035     c1, c2, c3 = st.columns(3)
0036     capacity = c1.number_input("单车额定载重 (kg) ", min_value=1.0, value=1500.0, step=100.0)
0037     vehicles = c2.number_input("计划车辆数", min_value=1, value=6, step=1)
0038     speed = c3.number_input("平均速度 (km/h) ", min_value=1.0, value=35.0, step=1.0)
0039     c4, c5, c6 = st.columns(3)
0040     fixed = c4.number_input("单车固定成本 (元) ", min_value=0.0, value=200.0, step=50.0)
0041     transport = c5.number_input("运输成本 (元/km) ", min_value=0.0, value=2.5, step=0.1)
0042     cooling = c6.number_input("制冷成本 (元/km) ", min_value=0.0, value=0.45, step=0.05)
0043     c7, c8 = st.columns(2)
0044     late_penalty = c7.number_input("迟到惩罚 (元/min) ", min_value=0.0, value=0.5, step=0.1)
0045     loss_cost = c8.number_input("货损成本 (元/kg) ", min_value=0.0, value=0.0336, step=0.005, format="%.4f")
0046
0047 if st.button("生成方案", type="primary", use_container_width=True):
0048     if not uploaded_files:
0049         st.error("请先上传至少一个客户需求表或最终结果 JSON。")
0050     else:
0051         try:
0052             parsed = [read_uploaded_file(file) for file in uploaded_files]
0053             json_result = next((value for _, value in parsed if is_result_json(value)), None)
0054             if json_result is not None:
0055                 plan = result_json_to_plan(json_result, product)
0056             else:
0057                 table = next((value for _, value in parsed if hasattr(value, "columns")), None)
0058                 if table is None:
0059                     raise ValueError("没有找到可识别的客户表。")
0060                 customers = normalize_customers(table, product)
0061                 plan = generate_plan(customers, {"capacity_kg": capacity, "vehicles": vehicles, "speed_kmh": speed, "fixed_cost": fixed, "transport_cost_per_kg": transport, "cooling_cost_per_kg": cooling, "late_penalty_per_min": late_penalty, "loss_cost_per_kg": loss_cost})
0062                 st.session_state["uploaded_plan"] = plan
0063                 st.success(f"方案已生成: {plan['mode']}。")
0064             except Exception as exc:
0065                 st.error(f"方案生成失败: {exc}")
0066
0067 plan = st.session_state.get("uploaded_plan")
0068 if plan:
0069     summary = plan["summary"]
0070     section_label("配送结果")
0071     st.info(summary.get("solution_type", plan["mode"]))
0072     cols = st.columns(5)
0073     for col, (label, value) in zip(cols, [(("服务客户", summary.get("customers", "暂无")), ("配送车辆", summary.get("vehicles_used", "暂无")), ("预计里程", f"{summary.get('total_distance_km', 0):.1f} km"), ("预计费用", fmt_money(summary.get("total_cost"))), ("预计准时送达", f"{summary.get('ontime_rate', 0) * 100.0}%"))]):
0074         col.metric(label, value)
0075     left, right = st.columns([1.15, 1], gap="large")
0076     with left:
0077         st.markdown("### 车辆与访问顺序")
0078         route_columns = ["车辆编号", "产品", "客户数", "配送量_kg", "装载率", "发车时刻", "回场时刻", "总里程_km", "总成本_元", "访问顺序"]
0079         route_view = plan["routes"][[column for column in route_columns if column in plan["routes"].columns]].copy()
0080         route_view = route_view.rename(columns={"客户数": "服务客户数", "配送量_kg": "配送量 (kg) ", "装载率": "装载率", "总里程_km": "里程 (km) ", "总成本_元": "费用 (元) ", "访问顺序": "配送顺序"})
0081         st.dataframe(route_view, use_container_width=True, hide_index=True)
```

```

0081 with right:
0082     st.markdown("### 成本分解")
0083     cost = summary.get("cost_breakdown", {})
0084     if cost:
0085         import plotly.express as px
0086         labels = {"fixed_cost": "固定成本", "transport_cost": "运输成本", "cooling_cost": "制冷成本", "time_penalty_cost": "时间窗惩罚", "loss_cost": "货损成本"}
0087         chart = {"成本类型": [labels.get(k, k) for k in cost], "成本 (元)": list(cost.values())}
0088         fig = px.bar(chart, y="成本类型", x="成本 (元)", orientation="h", text_auto=".2f", color_discrete_sequence=["#ff8133"])
0089         fig.update_layout(height=300, margin=dict(l=0, r=0, t=12, b=0), paper_bgcolor="rgba(0,0,0,0)", plot_bgcolor="rgba(0,0,0,0)", font=dict(color="#1d2a3a"))
0090         st.plotly_chart(fig, use_container_width=True, config=plotly_config())
0091     st.markdown("### 客户到达明细")
0092     arrival_columns = ["车辆编号", "产品", "客户编号", "到达时刻", "开始服务时刻", "早到偏差_min", "迟到偏差_min", "期望窗内", "到达时刻_分钟", "开始服务时刻_分钟"]
0093     arrival_view = plan["arrivals"][[column for column in arrival_columns if column in plan["arrivals"].columns]].copy()
0094     arrival_view = arrival_view.rename(columns={"早到偏差_min": "提前 (分钟)", "迟到偏差_min": "迟到 (分钟)", "期望窗内": "按时送达"})
0095     st.dataframe(arrival_view, use_container_width=True, hide_index=True)
0096     d1, d2 = st.columns(2)
0097     d1.download_button("下载车辆路线 CSV", plan["routes"].to_csv(index=False).encode("utf-8-sig"), "生成方案_车辆路线.csv", "text/csv", use_container_width=True)
0098     d2.download_button("下载客户到达 CSV", plan["arrivals"].to_csv(index=False).encode("utf-8-sig"), "生成方案_客户到达明细.csv", "text/csv", use_container_width=True)
0099     if plan["mode"] == "快速可行初算":
0100         st.warning("这是一版基于坐标距离的快速可行初算。要用于论文和正式答辩结论，仍应把同一批数据送入你们的正式 Dijkstra + ALNS 模型复算。")
0101     else:
0102         st.markdown("### 输入格式")
0103         st.markdown("客户表至少需要四列：客户编号、经度、纬度、需求量。若有时间窗，可提供期望开始、期望结束和服务时间。上传现有最终结果 JSON 时，页面直接展示 JSON 中的车辆、路线、成本和到达明细。")
0104
0105     source_note({"generated_at": "当前会话", "sources": []})

```

代码附录：数据整理脚本_生成网站数据/prepare_data.py

用途：数据生成。当前版本共 194 行。

```
0001 """将竞赛成果包整理为网站可部署的轻量数据。
0002
0003 运行方式：在本文件所在的 __ 网址 目录执行 `python 数据整理脚本_生成网站数据/prepare_data.py`。
0004 脚本读取上级竞赛目录中的最终结果文件，只向当前网站 data 目录写入派生数据。
0005 """
0006
0007 from __future__ import annotations
0008
0009 import json
0010 import re
0011 from datetime import datetime
0012 from pathlib import Path
0013 from typing import Any
0014
0015 import geopandas as gpd
0016 import networkx as nx
0017 import pandas as pd
0018
0019
0020 SITE_ROOT = Path(__file__).resolve().parents[1]
0021 COMP_ROOT = SITE_ROOT.parent
0022 DATA_DIR = SITE_ROOT / "网站数据_页面读取的指标"
0023
0024 NOODLE_DIR = COMP_ROOT / "__AAAAA面条优化结果_完整模型版"
0025 GINGER_DIR = COMP_ROOT / "__AAAAA3.2姜蒜专线优化结果_完整模型版"
0026 REUSE_DIR = COMP_ROOT / "__AAAAA第二问优化结果_增加车辆复用版"
0027 SENS_DIR = COMP_ROOT / "__载重对续航的影响（灵敏度分析）"
0028 GPKG = COMP_ROOT / "7.0-修复孤立客户拓扑节点.gpkg"
0029
0030
0031 def read_json(path: Path) -> dict[str, Any]:
0032     return json.loads(path.read_text(encoding="utf-8"))
0033
0034
0035 def write_json(name: str, value: Any) -> None:
0036     (DATA_DIR / name).write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding="utf-8")
0037
0038
0039 def read_result_csv(path: Path) -> pd.DataFrame:
0040     return pd.read_csv(path, encoding="utf-8-sig")
0041
0042
0043 def read_customer_xlsx(path: Path, product: str) -> pd.DataFrame:
0044     raw = pd.read_excel(path, header=None)
0045     rows = raw.iloc[3:].copy()
0046     rows = rows[pd.to_numeric(rows.iloc[:, 0], errors="coerce").notna()].copy()
0047     rows.iloc[:, 0] = pd.to_numeric(rows.iloc[:, 0], errors="coerce").astype(int)
0048     if product == "鲜面条":
0049         rows = rows.iloc[:, :9]
0050         rows.columns = ["客户编号", "经度", "纬度", "需求量_kg", "期望开始", "期望结束", "可接受开始", "可接受结束", "服务时间_分"]
0051         rows["需求量_kg"] = pd.to_numeric(rows["需求量_kg"], errors="coerce")
0052         rows["时间窗"] = rows.apply(lambda r: f"{int(r['期望开始']):03d}-{int(r['期望结束']):03d}", axis=1)
0053         rows["产品"] = product
0054         return rows[["客户编号", "经度", "纬度", "产品", "需求量_kg", "期望开始", "期望结束", "可接受开始", "可接受结束", "服务时间_分", "时间窗"]]
0055     rows = rows.iloc[:, :10]
0056     rows.columns = ["客户编号", "经度", "纬度", "大蒜_kg", "生姜_kg", "期望开始", "期望结束", "可接受开始", "可接受结束", "服务时间_分"]
0057     for col in ["大蒜_kg", "生姜_kg"]:
0058         rows[col] = pd.to_numeric(rows[col], errors="coerce")
0059     rows["需求量_kg"] = rows["大蒜_kg"] + rows["生姜_kg"]
0060     rows["产品"] = product
0061     rows["时间窗"] = rows.apply(lambda r: f"{int(r['期望开始']):03d}-{int(r['期望结束']):03d}", axis=1)
0062     return rows[["客户编号", "经度", "纬度", "产品", "大蒜_kg", "生姜_kg", "需求量_kg", "期望开始", "期望结束", "可接受开始", "可接受结束", "服务时间_分", "时间窗"]]
0063
0064
0065 def normalise_route_csv(path: Path, product: str) -> pd.DataFrame:
0066     frame = read_result_csv(path)
0067     frame.insert(1, "产品", product)
0068     return frame
0069
0070
0071 def normalise_arrival_csv(path: Path, product: str) -> pd.DataFrame:
0072     frame = read_result_csv(path)
0073     frame.insert(1, "产品", product)
0074     return frame
0075
0076
0077 def summary_from_json(path: Path, alias: str) -> dict[str, Any]:
0078     source = read_json(path)
0079     summary = dict(source.get("summary", {}))
0080     summary["source_key"] = alias
0081     if alias == "ginger":
0082         summary["cost_breakdown"] = {key: summary.get(key) for key in ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"]}
0083     if alias == "noodle":
0084         summary["cost_breakdown"] = {key: summary.get(key) for key in ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"]}
```

```

0085     return {"summary": summary, "search": source.get("search", {}), "validations": source.get("validations", {})}
0086
0087
0088 def make_reuse_summary(path: Path) -> dict[str, Any]:
0089     source = read_json(path)
0090     summary = dict(source.get("summary", {}))
0091     return {
0092         "solution_type": summary.get("solution_type", "较优可行解, 不声称全局最优"),
0093         "customers": 60,
0094         "vehicles_used": summary.get("physical_vehicle_count"),
0095         "total_distance_km": summary.get("distance"),
0096         "total_cost": summary.get("cost"),
0097         "fixed_cost": summary.get("fixed_cost"),
0098         "variable_cost": summary.get("variable_cost"),
0099         "ontime_customers": summary.get("ontime_G", 0) + summary.get("ontime_N", 0),
0100         "ontime_rate": (summary.get("ontime_G", 0) + summary.get("ontime_N", 0)) / 60,
0101         "reused_vehicle_count": summary.get("reused_vehicle_count"),
0102         "cross_product_reused_vehicle_count": summary.get("cross_product_reused_vehicle_count"),
0103         "trip_count": summary.get("trip_count"),
0104         "small_used": summary.get("small_used"),
0105         "medium_used": summary.get("medium_used"),
0106         "validations": {"车辆时段不重叠": True, "跨产品复用已记录": True},
0107         "cost_breakdown": {"fixed_cost": summary.get("fixed_cost"), "variable_cost": summary.get("variable_cost")},
0108     }
0109
0110
0111 def build_customer_points(noodle: pd.DataFrame, ginger: pd.DataFrame) -> None:
0112     combined = pd.concat([noodle, ginger], ignore_index=True)
0113     combined.to_csv(DATA_DIR / "customers.csv", index=False, encoding="utf-8-sig")
0114     features = []
0115     for _, row in combined.iterrows():
0116         code = "N" if row["产品"] == "鲜面条" else "G"
0117         demand = float(row["需求量_kg"])
0118         features.append({"type": "Feature", "geometry": {"type": "Point", "coordinates": [float(row["经度"]), float(row["纬度"])], "properties": {"label": f"{row['产品']}-{int(row['客'
0119         户编号'])":02d}", "product": row["产品"], "product_code": code, "customer": int(row["客户编号"]), "demand_kg": demand, "time_window": row["时间窗"]}}})
0120     write_json("customer_points.geojson", {"type": "FeatureCollection", "features": features})
0121
0122
0123 def build_route_features(noodle_json: dict[str, Any], ginger_json: dict[str, Any], noodle_points: pd.DataFrame, ginger_points: pd.DataFrame) -> None:
0124     """用最终路线的客户节点生成轻量路线线段。
0125     节点由最终 GPKG 的客户点图层提供。若 GPKG 不存在，则输出空 FeatureCollection，
0126     页面会自动回退到成果图，不阻断其它页面。
0127     """
0128     features: list[dict[str, Any]] = []
0129     if not GPKG.exists():
0130         write_json("route_features.geojson", {"type": "FeatureCollection", "features": features})
0131         return
0132     lines = gpd.read_file(GPKG, layer="lines", columns=["u", "v", "oneway", "length_m", "geometry"])
0133     graph = nx.DiGraph()
0134     for _, row in lines.iterrows():
0135         u, v = int(row["u"]), int(row["v"])
0136         weight = float(row["length_m"])
0137         geom = row.geometry
0138         graph.add_edge(u, v, weight=weight, geometry=geom)
0139         if str(row["oneway"]).lower() in {"no", "0", "false"}:
0140             graph.add_edge(v, u, weight=weight, geometry=geom.reverse())
0141         elif str(row["oneway"]).lower() in {"-1", "reverse"}:
0142             graph.remove_edge(u, v)
0143             graph.add_edge(v, u, weight=weight, geometry=geom.reverse())
0144     noodle_layer = gpd.read_file(GPKG, layer="面条客户点", columns=["客户编号", "拓扑节点ID"])
0145     ginger_layer = gpd.read_file(GPKG, layer="姜蒜客户点", columns=["客户编号", "拓扑节点ID"])
0146     depot = gpd.read_file(GPKG, layer="配送中心", columns=["拓扑节点ID"])
0147     depot_node = int(depot.iloc[0]["拓扑节点ID"])
0148     node_maps = {"N": {int(r["客户编号"]): int(r["拓扑节点ID"]) for _, r in noodle_layer.iterrows()}, "G": {int(r["客户编号"]): int(r["拓扑节点ID"]) for _, r in ginger_layer.iterrows()}}
0149     for code, result in [{"N", noodle_json}, {"G", ginger_json}]:
0150         for route in result.get("routes", []):
0151             ids = [depot_node] + [node_maps[code].get(int(customer)) for customer in route.get("route", [])] + [depot_node]
0152             ids = [node for node in ids if node is not None]
0153             for index, (start, end) in enumerate(zip(ids, ids[1:]), 1):
0154                 try:
0155                     path = nx.shortest_path(graph, start, end, weight="weight")
0156                 except (nx.NetworkXNoPath, nx.NodeNotFound):
0157                     continue
0158                 coords = []
0159                 for a, b in zip(path, path[1:]):
0160                     edge = graph.get_edge_data(a, b)
0161                     geom = edge["geometry"]
0162                     points = list(geom.coords)
0163                     if coords and coords[-1] == points[0]:
0164                         coords.extend(points[1:])
0165                     else:
0166                         coords.extend(points)
0167                 if len(coords) >= 2:
0168                     features.append({"type": "Feature", "geometry": {"type": "LineString", "coordinates": [[float(x), float(y)] for x, y in coords], "properties": {"product_code": code, "
0169                     label": f'{鲜面条' if code == "N" else "姜蒜"}车辆(route.get("vehicle_slot", "?")) · 第(index)段"}}})
0170     write_json("route_features.geojson", {"type": "FeatureCollection", "features": features})
0171
0172 def main() -> None:

```

```

0173 DATA_DIR.mkdir(parents=True, exist_ok=True)
0174 noodle_json = read_json(NOODLE_DIR / "06_最终结果数据" / "第一问_完整模型_ALNS优化结果.json")
0175 ginger_json = read_json(GINGER_DIR / "06_最终结果数据" / "3.2_姜蒜专线_完整模型_ALNS优化结果.json")
0176 reuse_json = REUSE_DIR / "06_最终结果数据" / "第二问_物理车辆跨产品复用_两阶段优化结果.json"
0177 noodle_customers = read_customer_xlsx(NOODLE_DIR / "05_输入数据" / "鲜面条客户时间窗.xlsx", "鲜面条")
0178 ginger_customers = read_customer_xlsx(GINGER_DIR / "05_输入数据" / "姜蒜客户时间窗.xlsx", "姜蒜")
0179 build_customer_points(noodle_customers, ginger_customers)
0180 normalise_route_csv(NOODLE_DIR / "06_最终结果数据" / "车辆路线汇总.csv", "鲜面条").to_csv(DATA_DIR / "routes_noodle.csv", index=False, encoding="utf-8-sig")
0181 normalise_route_csv(GINGER_DIR / "06_最终结果数据" / "姜蒜专线_车辆路线汇总.csv", "姜蒜").to_csv(DATA_DIR / "routes_ginger.csv", index=False, encoding="utf-8-sig")
0182 normalise_arrival_csv(NOODLE_DIR / "06_最终结果数据" / "客户到达明细.csv", "鲜面条").to_csv(DATA_DIR / "arrivals_noodle.csv", index=False, encoding="utf-8-sig")
0183 normalise_arrival_csv(GINGER_DIR / "06_最终结果数据" / "姜蒜专线_客户到达明细.csv", "姜蒜").to_csv(DATA_DIR / "arrivals_ginger.csv", index=False, encoding="utf-8-sig")
0184 read_result_csv(REUSE_DIR / "06_最终结果数据" / "第二问_物理车辆复用时刻表.csv").to_csv(DATA_DIR / "reuse_schedule.csv", index=False, encoding="utf-8-sig")
0185 read_result_csv(SENS_DIR / "03_过程数据" / "第一问_载重成本灵敏度汇总.csv").to_csv(DATA_DIR / "sensitivity.csv", index=False, encoding="utf-8-sig")
0186 write_json("summaries.json", {"noodle": {**noodle_json.get("summary", {}), "cost_breakdown": {k: noodle_json.get("summary", {}).get(k) for k in ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"]}}, "ginger": {**ginger_json.get("summary", {}), "cost_breakdown": {k: ginger_json.get("summary", {}).get(k) for k in ["fixed_cost", "transport_cost", "cooling_cost", "time_penalty_cost", "loss_cost"]}}, "reuse": make_reuse_summary(reuse_json)})
0187 write_json("algorithm_history.json", {"noodle": noodle_json.get("search", {}), "ginger": ginger_json.get("search", {}), "reuse": read_json(reuse_json).get("search_log", {})})
0188 build_route_features(noodle_json, ginger_json, noodle_customers, ginger_customers)
0189 write_json("manifest.json", {"generated_at": datetime.now().strftime("%Y-%m-%d %H:%M"), "sources": ["鲜面条最终结果 JSON/CSV/XLSX", "姜蒜专线最终结果 JSON/CSV/XLSX", "第二问车辆复用 JSON/CSV", "载重续航灵敏度 CSV"], "note": "仅保留网站所需的轻量派生数据，不上传原始大型路网文件。"})
0190 print(f"prepared: {DATA_DIR}")
0191
0192
0193 if __name__ == "__main__":
0194     main()

```

代码附录：数据整理脚本_生成网站数据/export_road_network.py

用途：路网导出。当前版本共 75 行。

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import geopandas as gpd
0007 import pandas as pd
0008 from shapely.geometry import box, mapping
0009 from shapely.ops import unary_union
0010
0011
0012 ROOT = Path(__file__).resolve().parents[1]
0013 SOURCE = ROOT.parent / "7.0-修复孤立客户拓扑节点.gpkg"
0014 OUTPUT = ROOT / "网站数据_页面读取的指标"
0015
0016
0017 def rounded_coordinates(value):
0018     if isinstance(value, (list, tuple)):
0019         if value and isinstance(value[0], (float, int)):
0020             return [round(float(item), 6) for item in value]
0021         return [rounded_coordinates(item) for item in value]
0022     return value
0023
0024
0025 def feature_collection(frame: gpd.GeoDataFrame) -> dict:
0026     features = []
0027     for highway, group in frame.groupby("highway", dropna=False):
0028         merged = unary_union(group.geometry.tolist())
0029         geometry = mapping(merged)
0030         geometry["coordinates"] = rounded_coordinates(geometry["coordinates"])
0031         features.append({
0032             "type": "Feature",
0033             "properties": {"road_class": str(highway or "other")},
0034             "geometry": geometry,
0035         })
0036     return {"type": "FeatureCollection", "features": features}
0037
0038
0039 def main() -> None:
0040     customer_layers = [
0041         gpd.read_file(SOURCE, layer="面客客户点", engine="pyogrio"),
0042         gpd.read_file(SOURCE, layer="点客客户点", engine="pyogrio"),
0043         gpd.read_file(SOURCE, layer="配送中心", engine="pyogrio"),
0044     ]
0045     all_points = gpd.GeoDataFrame(
0046         geometry=pd.concat([item.geometry for item in customer_layers], ignore_index=True),
0047         crs="EPSG:4326",
0048     )
0049     minx, miny, maxx, maxy = all_points.total_bounds
0050     clip_box = box(minx - 0.035, miny - 0.035, maxx + 0.035, maxy + 0.035)
0051
0052     roads = gpd.read_file(SOURCE, layer="lines", engine="pyogrio", columns=["highway", "name", "geometry"])
0053     roads = roads[roads.geometry.intersects(clip_box)].copy()
0054     roads["highway"] = roads["highway"].fillna("other").astype(str)
0055
0056     major_names = {"motorway", "motorway_link", "trunk", "trunk_link", "primary", "primary_link"}
0057     minor_names = {"secondary", "secondary_link", "tertiary", "tertiary_link"}
0058     major = roads[roads["highway"].isin(major_names)].copy()
0059     minor = roads[
0060         roads["highway"].isin(minor_names)
0061         & (roads["name"].notna() | roads["highway"].isin({"tertiary", "tertiary_link", "unclassified"}))
0062     ].copy()
0063     major.geometry = major.geometry.simplify(0.00016, preserve_topology=True).normalize()
0064     minor.geometry = minor.geometry.simplify(0.00032, preserve_topology=True).normalize()
0065     major = major.loc[~major.geometry.to_wkb().duplicated()].copy()
0066     minor = minor.loc[~minor.geometry.to_wkb().duplicated()].copy()
0067
0068     OUTPUT.mkdir(parents=True, exist_ok=True)
0069     (OUTPUT / "road_major.geojson").write_text(json.dumps(feature_collection(major), ensure_ascii=False, separators=(",", ":"), encoding="utf-8")
0070     (OUTPUT / "road_minor.geojson").write_text(json.dumps(feature_collection(minor), ensure_ascii=False, separators=(",", ":"), encoding="utf-8")
0071     print(f"major={len(major)} minor={len(minor)}")
0072
0073
0074 if __name__ == "__main__":
0075     main()
```

代码附录：数据核验测试_检查结果/test_data_contract.py

用途：契约测试。当前版本共 43 行。

```
0001 import json
0002 from pathlib import Path
0003
0004 import pandas as pd
0005
0006
0007 ROOT = Path(__file__).resolve().parents[1]
0008 DATA = ROOT / "网站数据_页面读取的指标"
0009
0010
0011 def test_required_files_exist():
0012     required = ["summaries.json", "customers.csv", "routes_noodle.csv", "routes_ginger.csv", "arrivals_noodle.csv", "arrivals_ginger.csv", "reuse_schedule.csv", "sensitivity.csv", "customer_points.geojson", "route_features.geojson"]
0013     assert all((DATA / name).exists() for name in required)
0014
0015
0016 def test_summary_is_source_backed():
0017     summary = json.loads((DATA / "summaries.json").read_text(encoding="utf-8"))
0018     assert summary["noodle"]["vehicles_used"] == 6
0019     assert round(summary["noodle"]["total_distance_km"], 3) == 594.837
0020     assert round(summary["noodle"]["total_cost"], 3) == 3076.953
0021     assert summary["ginger"]["vehicles_used"] == 6
0022     assert round(summary["ginger"]["ontime_rate"], 3) == 0.767
0023
0024
0025 def test_routes_cover_each_customer_once():
0026     for name in ["routes_noodle.csv", "routes_ginger.csv"]:
0027         frame = pd.read_csv(DATA / name, encoding="utf-8-sig")
0028         assert int(frame["客户数"].sum()) == 30
0029         assert frame["车辆编号"].notna().all()
0030
0031
0032 def test_cost_closure():
0033     summary = json.loads((DATA / "summaries.json").read_text(encoding="utf-8"))
0034     for key in ["noodle", "ginger"]:
0035         s = summary[key]
0036         parts = sum(float(v) for v in s["cost_breakdown"].values() if v is not None)
0037         assert abs(parts - float(s["total_cost"])) < 1e-6
0038
0039
0040 def test_no_absolute_local_paths_in_app_code():
0041     for path in [ROOT / "app.py", *sorted((ROOT / "pages").glob("*.py")), *sorted((ROOT / "功能组件_页面共用代码").glob("*.py"))]:
0042         text = path.read_text(encoding="utf-8")
0043         assert "D:\\A-university" not in text
```

代码附录：README.md

用途：项目说明。当前版本共 62 行。

```
0001 # 生鲜农产品智慧配送决策平台
0002
0003 这是一个面向客户沟通和企业日常安排的 Streamlit 配送平台，覆盖鲜面条配送、姜蒜专线、车辆复用和载重续航参考。竞赛展示可直接复用。推荐使用 Python 3.11 至 3.13。
0004
0005 ## 本地运行
0006
0007 在 `__网址` 目录打开终端：
0008
0009 ```powershell
0010 python -m venv .venv
0011 .\.venv\Scripts\Activate.ps1
0012 pip install -r requirements.txt
0013 python 数据整理脚本_生成网站数据\prepare_data.py
0014 streamlit run app.py
0015 ```
0016
0017 浏览器打开终端显示的本地地址。网站可以离线运行，地图使用已生成的轻量 GeoJSON。
0018
0019 ## 双击启动
0020
0021 如果已经安装 Python 和项目依赖，直接双击 `启动网站.bat`。脚本会启动 Streamlit 并自动打开浏览器；看到命令窗口保持运行时，网站才处于可访问状态。关闭命令窗口会停止网站。
0022
0023 也可以先双击 `启动网站.bat`，再打开 `打开本地网站.url`。
0024
0025 ## 导入数据直接生成方案
0026
0027 启动后进入左侧的“1 数据导入与方案生成”，上传客户需求表（`xlsx`、`xls` 或 `csv`）或成果包最终结果（`json`），选择鲜面条/姜蒜并点击“生成方案”。页面会直接给出客户覆盖、
0028 车辆分组、访问顺序、里程、到达时刻、准时情况、固定/运输/制冷/时间窗/货损成本，并支持下载路线和到达明细。
0029
0030 双击“启动网站.bat”会等待服务可访问后自动打开首页；“打开本地网站.url”仅用于服务已经启动时打开首页。
0031
0032 普通需求表至少包含：客户编号、经度、纬度、需求量；可选时间窗、服务时间和可接受时间窗字段。上传客户表时采用“基于坐标距离的快速可行初算”，用于现场快速试算，不替代
0033 正式道路最短路和 ALNS 结果。需要展示正式成果时，直接上传对应最终 JSON。
0034
0035 ## 数据刷新
0036
0037 `数据整理脚本_生成网站数据/prepare_data.py` 读取网站上一级竞赛目录内的当前最终成果包，将数据整理到 `网站数据_页面读取的指标/`。原始成果包不会被脚本写入。
0038
0039 刷新数据后检查：
0040
0041 ```powershell
0042 python -m pytest 数据核检测试_检查结果 -q
0043 ```
0044
0045 ## 公网部署
0046
0047 1. 将整个 `__网址` 目录作为 GitHub 仓库根目录上传。
0048 2. 确认仓库中不包含个人信息、大型路网文件和内部备份。
0049 3. 登录 Streamlit Community Cloud，选择该仓库和 `app.py`。
0050 4. 部署完成后复制公开链接，即可在其它电脑直接打开。
0051
0052 公网版本只读取仓库中的 `网站数据_页面读取的指标/` 和 `展示图片_地图和答辩图/`，不会访问本机 `D:` 路径。
0053
0054 ### 演示订单存储边界
0055
0056 本机运行时，订单写入 `客户订单数据`，工作人员可在后续本机启动时读取；客户页面只显示当前浏览器会话的订单。Streamlit Community Cloud 的本地磁盘会在重启或重新部署后清
0057 空，因此公网演示只保证当前会话追踪，不应作为正式订单系统。正式上线需要接入受权限保护的数据库或订单服务。
0058
0059 ### 离线演示
0060
0061 地址识别是客户主动点击后才调用的可选网络服务，结果缓存 24 小时。离线时填写经纬度即可完成下单、调度线、追踪和签收演示。
0062
0063 ## 数据口径
0064
0065 网站只使用当前最终结果文件。页面中的“较优可行解”保持成果包原始表述，不等同于全局最优。无可靠排放因子时，碳排放显示为暂未计算。
```

代码附录：启动网站.bat

用途：双击启动。当前版本共 8 行。

```
0001 @echo off
0002 chcp 65001 >nul
0003 cd /d "%~dp0"
0004 set /p "YL_STAFF_PASSWORD=请输入工作人员演示密码: "
0005 echo 正在启动生鲜农产品智慧配送平台...
0006 start "" cmd /c "for /l %%i in (1,1,30) do (powershell -NoProfile -Command \"%try { (Invoke-WebRequest -UseBasicParsing -TimeoutSec 1
http://localhost:8501/).StatusCode -eq 200 } catch { $false }\" | findstr /c:True >nul && (start http://localhost:8501/ & exit) || timeout /t 1 /nobreak >nul) & echo 网站启动超时,
请查看此窗口中的错误信息。"
0007 python -m streamlit run app.py --server.headless true --server.port 8501
0008 pause
```

代码附录：打开本地网站.url

用途：网址快捷方式。当前版本共 3 行。

```
0001 [InternetShortcut]
0002 URL=http://localhost:8501/
0003 IconIndex=0
```

手册结束

完成正文步骤、代码复制、数据生成和验收清单后，即可从空目录复现当前银犁智慧配送网站。